Learn with Me Daily

Java

Java Practice Question Bank

Learn
Build
Grow

SECTION 1 – JAVA BASICS (1-30)

Beginner Level Problems

1. Hello World Program
2. Print User Details
3. Addition of Two Numbers
4. Swap Two Numbers
5. Even or Odd
6. Largest of 3 Numbers
7. Positive or Negative
8. Leap Year Check
9. Simple Interest
10. Compound Interest
11. Area of Circle
12. Area of Rectangle
13. Celsius to Fahrenheit
14. ASCII Value of Character
15. Calculator Using Switch
16. Voting Eligibility
17. Grade Calculator
18. Reverse Number
19. Palindrome Number
20. Armstrong Number
21. Prime Number
22. Fibonacci Series
23. Factorial Program
24. Sum of Digits
25. Count Digits
26. GCD Program
27. LCM Program
28. Multiplication Table
29. Menu Driven Program
30. ATM Simulation

TIPS TO SOLVE

- Understand the problem statement carefully.
- Break the problem into small steps.
- Try on paper before coding.
- Use logic and formula where required.
- Test your code with different inputs.
- Practice daily and stay consistent.

Learn with Me Daily

Java

Java Practice Question Bank

Learn
Build
Grow

SECTION 2 – LOOP & PATTERN PROBLEMS (31-60)

Pattern Printing & Loop Based Problems

31. Square Star Pattern
32. Right Triangle Pattern
33. Left Triangle Pattern
34. Pyramid Pattern
35. Inverted Pyramid
36. Diamond Pattern
37. Hollow Square Pattern
38. Hollow Pyramid
39. Butterfly Pattern
40. Pascal Triangle
41. Number Triangle
42. Floyd Triangle
43. Binary Triangle
44. Character Triangle
45. Sandglass Pattern
46. Hourglass Pattern
47. X Pattern
48. Zig-Zag Pattern
49. Hollow Diamond
50. Rhombus Pattern
51. Reverse Number Triangle
52. Prime Pyramid
53. Palindrome Pyramid
54. Alphabet Pyramid
55. Spiral Pattern
56. Matrix Spiral Pattern
57. Staircase Pattern
58. Arrow Pattern
59. Checkerboard Pattern
60. Border Stars Pattern

TIPS TO MASTER PATTERNS

- Understand the pattern logic before writing code.
- Identify rows and columns carefully.
- Use dry run on paper.
- Observe symmetry in patterns.
- Practice more to improve speed.
- Break the pattern into small parts.

Learn with Me Daily

Java Practice Question Bank

Learn
Build
Grow

SECTION 3 – ARRAYS PROBLEMS (61-90)

Master Arrays with Logic & Practice

61. Array Input Output
62. Array Sum
63. Largest Element in Array
64. Smallest Element in Array
65. Second Largest Element
66. Reverse Array
67. Sort Array Ascending
68. Sort Array Descending
69. Linear Search
70. Binary Search
71. Frequency of Elements
72. Remove Duplicates
73. Merge Two Arrays
74. Left Rotate Array
75. Right Rotate Array
76. Find Missing Number
77. Find Duplicate Elements
78. Find Unique Elements
79. Pair with Given Sum
80. Kadane's Algorithm
81. Maximum Product Subarray
82. Equilibrium Index
83. Majority Element
84. Move Zeros to End
85. Union of Two Arrays
86. Intersection of Two Arrays
87. Matrix Addition
88. Matrix Multiplication
89. Matrix Transpose
90. Spiral Matrix Traversal

TIPS TO SOLVE ARRAY PROBLEMS

- Understand the problem carefully.
- Visualize the array on paper.
- Identify the pattern or logic.
- Use simple examples first.
- Dry run your code.
- Optimize for time & space.
- Practice consistently.

Learn with Me Daily

Java

Java Practice Question Bank

Learn
Build
Grow

SECTION 4 – STRING PROBLEMS (91-120)

Master Strings with Logic & Practice

91. Reverse a String
92. Palindrome String
93. Count Vowels
94. Count Consonants
95. Count Words
96. Remove Spaces
97. Toggle Case
98. Uppercase to Lowercase
99. Lowercase to Uppercase
100. Check Anagram Strings
101. Find Duplicate Characters
102. Character Frequency
103. First Non-Repeating Character
104. First Repeating Character
105. Remove Duplicate Characters
106. Sort Characters in String
107. Reverse Words in Sentence
108. Check Pangram
109. Remove Vowels
110. String Compression
111. Run Length Encoding
112. Check Rotation of String
113. Check Subsequence
114. Longest Common Prefix
115. Longest Palindrome Substring
116. Longest Unique Substring
117. Valid Parentheses
118. Roman to Integer
119. Integer to Roman
120. Group Anagrams

> Strings are the foundation of many real-world problems.
> Practice daily, master logics, crack interviews!

TIPS TO MASTER STRING PROBLEMS

- Understand the problem carefully.
- Break the string into small parts.
- Think of edge cases.
- Use built-in functions wisely.
- Test with different inputs.
- Practice consistently.

Learn with Me Daily

Java

Java Practice Question Bank

Learn
Build
Grow

SECTION 5 – OOP & DSA PROBLEMS (121-150)

Strengthen OOP Concepts & DSA Skills

121. Student Class
122. Employee Class
123. Bank Account Class
124. Constructor Overloading
125. Method Overloading
126. Method Overriding
127. Single Inheritance
128. Multilevel Inheritance
129. Encapsulation Example
130. Abstraction Example
131. Interface Example
132. Runtime Polymorphism
133. Singleton Class
134. Immutable Class
135. Object Cloning
136. Stack Using Array
137. Queue Using Array
138. Linked List Insertion
139. Reverse Linked List
140. Binary Tree Traversal
141. Bubble Sort
142. Selection Sort
143. Insertion Sort
144. Merge Sort
145. Quick Sort
146. DFS Traversal
147. BFS Traversal
148. Fibonacci Using DP
149. 0/1 Knapsack
150. Sudoku Solver

Master OOP.
Strengthen DSA.
Crack Interviews.
Build Your Future!

TIPS TO MASTER OOP & DSA

- Understand concepts before coding.
- Focus on logic building.
- Practice consistently.

- Solve by yourself first.
- Analyze time & space complexity.

- Revise regularly.
- Solve previous year interview questions.
- Keep building projects.

Learn with Me Daily

Java Practice Question Bank

Learn
Build
Grow

SECTION 1 – JAVA BASICS (1-30)

Beginner Level Problems

1. Hello World Program

Code:

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Explanation:

This is the simplest Java program.
It prints "Hello, World!" on the screen.

Output:

```
Hello, World!
```

2. Print User Details

Code:

```
import java.util.Scanner;
public class UserDetails {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter your name: ");
        String name = sc.nextLine();
        System.out.print("Enter your age: ");
        int age = sc.nextInt();
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        sc.close();
    }
}
```

Explanation:

- Takes name (string) and age (integer) from the user.
- Displays the entered details.

Output:

```
Enter your name:
John
Enter your age:
20
Name: John
Age: 20
```

3. Addition of Two Numbers

Code:

```
import java.util.Scanner;
public class Addition {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter first number: ");
        int a = sc.nextInt();
        System.out.print("Enter second number: ");
        int b = sc.nextInt();
        int sum = a + b;
        System.out.println("Sum: " + num);
        sc.close();
    }
}
```

Explanation:

- Reads two numbers from the user.
- Adds them and displays the sum.

Output:

```
Enter first number:
10
Enter second number:
20
Sum: 30
```

4. Swap Two Numbers

Code:

```
import java.util.Scanner;
public class SwapNumbers {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter first number: ");
        int a = sc.nextInt();
        System.out.print("Enter second number: ");
        int b = sc.nextInt();
        int temp = a;
        a = b;
        b = temp;
        System.out.println("After Swapping:");
        System.out.println("First number: " + a);
        System.out.println("Second number: " + b);
        sc.close();
    }
}
```

Explanation:

- Swaps the values of two numbers using a temporary variable.

Output:

```
Enter first number:
5
Enter second number:
10
After Swapping:
First number: 10
Second number: 5
```

5. Even or Odd

Code:

```
import java.util.Scanner;
public class EvenOdd {
    public static void main(String[] args) {
        Scanner sc = newScanner(System.in);
        System.out.print("Enter a number: ");
        int n = sc.nextInt();
        if (n % 2 == 0)
            System.out.println(n + " is Even");
        else
            System.out.println(n + " is Odd");
        sc.close();
    }
}
```

Explanation:

- Checks if a number is divisible by 2.
- If yes → Even otherwise → Odd.

Output:

```
Enter a number:
7
7 is Odd
```

Learn with Me Daily

Java Practice Question Bank

Learn Build Grow

SECTION 1 – JAVA BASICS (1-30)

→ Beginner Level Problems ←

6. Largest of 3 Numbers

Code:

```
import java.util.Scanner;
public class LargestOfThree {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter first number: ");
      int a = sc.nextInt();
      System.out.print("Enter second number: ");
      int b = sc.nextInt();
      System.out.print("Enter third number: ");
      int c = sc.nextInt();
      int largest = a;
      if (b > largest) largest = b;
      if (c > largest) largest = c;
      System.out.println("Largest number is: " + largest);
      sc.close();
   }
}
```

Explanation:

- Takes three numbers from the user.
- Compares them using if statements.
- Displays the largest number.

Output:

```
Enter first number: 12
Enter second number: 25
Enter third number: 7

Largest number is: 25
```

7. Positive or Negative

Code:

```
import java.util.Scanner;
public class PositiveNegative {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a number: ");
      int n = sc.nextInt();
      if (n >= 0)
          System.out.println(n + " is Positive.");
      else
          System.out.println(n + " is Negative.");
   } sc.close();
}
```

Explanation:

- Reads a number.
- If number is greater than or equal to 0, it is Positive.
- Otherwise, it is Negative.

Output:

```
Enter a number: -9
-9 is Negative.
```

8. Leap Year Check

Code:

```
import java.util.Scanner;
public class LeapYear {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter year: ");
      int year = sc.nextInt();
      boolean leap = (year % 400 == 0) ||
                     (year % 4 == 0 && year % 100 != 0);
      if (leap)
          System.out.println(year + " is a Leap Year.");
      else
          System.out.println(year + " is Not a Leap Year.");
      sc.close();
   }
}
```

Explanation:

- A year is leap if:
  - Divisible by 400 OR
  - Divisible by 4 but not by 100.
- Prints the result.

Output:

```
Enter year: 2024
2024 is a Leap Year.
```

9. Simple Interest

Code:

```
import java.util.Scanner;
public class SimpleInterest {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter Principal (P): ");
      double P = sc.nextDouble();
      System.out.print("Enter Rate (R): ");
      double R = sc.nextDouble();
      System.out.print("Enter Time (T): ");
      double T = sc.nextDouble();
      double SI = (P * R * T) / 100;
      System.out.println("Simple Interest = " + SI);
      sc.close();
   }
}
```

Explanation:

- Simple Interest formula:
  SI = (P × R × T) / 100
  where,
  P = Principal
  R = Rate of interest
  T = Time (in years)
- Calculates and prints simple interest.

Output:

```
Enter Principal (P): 10000
Enter Rate (R): 5
Enter Time (T): 2

Simple Interest = 1000.0
```

Tips: • Read the question carefully. • Understand the logic before coding. • Practice daily to improve.

10. Factorial of a Number

Code:

```
import java.util.Scanner;
public class Factorial {
   public static void main(String[] args) {
       Scanner sc = new Scanner(System.in);
       System.out.print("Enter a number: ");
       int n = sc.nextInt();
       long fact = 1;
       for(int i = 1; i <= n; i++) {
           fact *= i;
       }
       System.out.println("Factorial of " + n +
           " is: " + fact);
       sc.close();
   }
}
```

Explanation:

- Reads a number n.
- Multiplies all numbers from 1 to n.
- Prints the factorial of the number.

Output:

```
Enter a number: 5
Factorial of 5 is: 120
```

11. Fibonacci Series

Code:

```
import java.util.Scanner;
public class Fibonacci {
   public static void main(String[] args) {
       Scanner sc = new Scanner(System.in);
       System.out.print("Enter number of terms: ");
       int n = sc.nextInt();
       int a = 0, b = 1, next;
       System.out.print("Fibonacci Series: ");
       for(int i = 1; i <= n; i++) {
           System.out.print(a + " ");
           next = a + b;
           a = b;
           b = next;
       }
       sc.close();
   }
}
```

Explanation:

- Reads number of terms n.
- Starts with 0 and 1.
- Each next term is sum of previous two terms.
- Prints the series.

Output:

```
Enter number of terms: 7
Fibonacci Series: 0 1 1 2 3 5 8
```

12. Prime Number Check

Code:

```
import java.util.Scanner;
public class PrimeCheck {
   public static void main(String[] args) {
       Scanner sc = new Scanner(System.in);
       System.out.print("Enter a number: ");
       int n = sc.nextInt();
       boolean isPrime = true;
       if (n <= 1) {
           isPrime = false;
       } else {
           for(int i = 2; i <= Math.sqrt(n); i++) {
               if(n % i == 0) {
                   isPrime = false;
                   break;
               }
           }
       }
       if(isPrime)
           System.out.println(n + " is Prime.");
       else
           System.out.println(n + " is Not Prime.");
       sc.close();
   }
}
```

Explanation:

- Reads a number n.
- Checks divisibility from 2 to √n.
- If divisible by any number other than 1 and itself, it is not prime.
- Prints the result.

Output:

Example 1

```
Enter a number: 29
29 is Prime.
```

Example 2

```
Enter a number: 18
18 is Not Prime.
```

Tip: Understand the logic, practice more examples, and try solving without looking at the code.

13. Sum of Digits

Code:

```
import java.util.Scanner;
public class SumOfDigits {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int n = sc.nextInt();
        int sum = 0;
        while (n > 0) {
            int digit = n % 10;
            sum += digit;
            n /= 10;
        }
        System.out.println("Sum of digits: " + sum);
        sc.close();
    }
}
```

Explanation:

- Reads a number n.
- Extracts each digit using n % 10.
- Adds the digit to sum.
- Removes the last digit using n / 10.
- Repeats until n becomes 0.
- Prints the sum.

Output:

```
Enter a number: 12345
Sum of digits: 15
```

14. Reverse a Number

Code:

```
import java.util.Scanner;
public class ReverseNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int n = sc.nextInt();
        int rev = 0;
        while (n > 0) {
            int digit = n % 10;
            rev = rev * 10 + digit;
            n /= 10;
        }
        System.out.println("Reversed number: " + rev);
        sc.close();
    }
}
```

Explanation:

- Reads a number n.
- Extracts the last digit using n % 10.
- Builds the reversed number by rev = rev * 10 + digit.
- Removes the last digit using n / 10.
- Prints the reversed number.

Output:

```
Enter a number: 4567
Reversed number: 7654
```

15. Armstrong Number

Code:

```
import java.util.Scanner;
public class ArmstrongNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int n = sc.nextInt();
        int temp = n, sum = 0, digits = 0;
        int original = n;
        while (temp > 0) {
            temp /= 10;
            digits++;
        }
        temp = n;
        while (temp > 0) {
            int digit = temp % 10;
            sum += Math.pow(digit, digits);
            temp /= 10;
        }
        if (sum == original)
            System.out.println(n + " is an Armstrong Number.");
        else
            System.out.println(n + " is Not an Armstrong Number.");
        sc.close();
    }
}
```

Explanation:

- Counts the number of digits.
- Raises each digit to the power of total digits.
- Adds them up.
- If sum equals the original number, it is an Armstrong number.

Output:

Example 1

```
Enter a number: 153
153 is an Armstrong Number.
```

Example 2

```
Enter a number: 123
123 is Not an Armstrong Number.
```

16. Palindrome Number

Code:

```
import java.util.Scanner;
public class PalindromeNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int n = sc.nextInt();
        int original = n, rev = 0;
        while (n > 0) {
            int digit = n % 10;
            rev = rev * 10 + digit;
            n /= 10;
        }
        if (rev == original)
            System.out.println(original + " is a Palindrome Number.");
        else
            System.out.println(original + " is Not a Palindrome Number.");
        sc.close();
    }
}
```

Explanation:

- Reverses the number.
- Compares the reversed number with the original.
- If both are same, it is a palindrome.
- Otherwise, it is not.

Output:

Example 1

```
Enter a number: 121
121 is a Palindrome Number.
```

Example 2

```
Enter a number: 1234
1234 is Not a Palindrome Number.
```

Tip: Practice these problems daily to strengthen your logic and problem-solving skills!

17. GCD of Two Numbers

Code:

```
import java.util.Scanner;
public class GCD {
  public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter first number: ");
      int a = sc.nextInt();
      System.out.print("Enter second number: ");
      int b = sc.nextInt();
      int gcd = gcd(a, b);
      System.out.println("GCD of " + a + " and " + b
            + " is:  " + gcd);
      sc.close();
  }
  public static int gcd(int x, int y) {
     while (y != 0) {
          int temp = y;
          y = x % y;
          x = temp;
     }
     return x;
  }
}
```

Explanation:

- Reads two numbers a and b.
- Uses Euclidean Algorithm: gcd(a, b) = gcd(b, a % b) until b becomes 0.
- When b = 0, a is the GCD.
- Prints the GCD.

Output:

```
Enter first number: 48
Enter second number: 18
GCD of 48 and 18 is: 6
```

18. LCM of Two Numbers

Code:

```
import java.util.Scanner;
public class LCM {
  public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter first number: ");
      int a = sc.nextInt();
      System.out.print("Enter second number: ");
      int b = sc.nextInt();
      int gcd = gcd(a, b);
      int lcm = (a / gcd) * b;
      System.out.println("LCM of " + a + " and " + b
            +   + " is: " + lcm);
      sc.close();
  }
  public static int gcd(int x, int y) {
     while (y != 0) {
          int temp = y;
          y = x % y;
          x = temp;
      }
      return x;
  }
}
```

Explanation:

- Reads two numbers a and b.
- First finds GCD using Euclidean Algorithm.
- Then uses formula: LCM(a, b) = (a / gcd) * b
- Prints the LCM.

Output:

```
Enter first number: 12
Enter second number: 15
LCM of 12 and 15 is: 60
```

19. Count Vowels in a String

Code:

```
import java.util.Scanner;
public class CountVowels {
  public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a string: ");
      String str = sc.nextLine();
      int count = 0;
      str = str.toLowerCase();
      for(int i = 0; i < str.length(); i++) {
         char ch = str.charAt(i);
         if (ch == 'a' || ch == 'e' || ch == 'i'
              || ch != 'o' || ch == 'u') {
             count++;
         }
      }
      System.out.println("Number of vowels: " + count);
      sc.close();
  }
}
```

Explanation:

- Reads a string from user.
- Converts string to lower case to handle both cases.
- Checks each character, if it is a vowel (a, e, i, o, u), increases count.
- Prints total vowels.

Output:

```
Enter a string: Hello World
Number of vowels: 3
```

20. Count Consonants in a String

Code:

```
Code:
import java.util.Scanner;
public class CountConsonants {
  public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a string: ");
      String str = sc.nextLine();
      int count = 0;
      str = str.toLowerCase();
      for(int i = 0; i < str.length(); i++) {
         char ch = str.charAt(i);
         if (ch >= 'a' && ch <= 'z') {
            if (ch != 'a' && ch != 'e' && ch != 'i'
                && ch != 'o' && ch != 'u') {
                 count++;
            }
         }
      }
      System.out.println("Number of consonants: " + count);
      sc.close();
  }
}
```

Explanation:

- Reads a string from user.
- Converts to lower case.
- For each character, if it is a letter (a–z) and not a vowel, then it is a consonant.
- Counts and prints consonants.

Output:

```
Enter a string: Programming
Number of consonants: 6
```

⭐ **Tip:** Practice regularly and try modifying the code to solve variations!

21. Check Anagram

Code:

```
import java.util.Arrays;
import java.util.Scanner;
public class AnagramCheck {
  public static void main(String[] args) {
     Scanner sc = new Scanner(System.in);
     System.out.print("Enter first string: ");
     String s1 = sc.nextLine().toLowerCase();
     System.out.print("Enter second string: ");
     String s2 = sc.nextLine().toLowerCase();
     if (s1.length() != s2.length()) {
         System.out.println("Not Anagrams");
     } else {
         char[] a = s1.toCharArray();
         char[] b = s2.toCharArray();
         Arrays.sort(a);
         Arrays.sort(b);
         if (Arrays.equals(a, b))
             System.out.println("Anagrams");
         else
             System.out.println("Not Anagrams");
     }
     sc.close();
  }
}
```

Explanation:

- Reads two strings.
- Converts to lower case.
- If lengths differ → not anagrams.
- Sorts characters of both strings.
- If sorted strings are equal → anagrams, else not.
- Prints the result.

Output:

Example 1

```
Enter first string: listen
Enter second string: silent
Anagrams
```

Example 2

```
Enter first string: hello
Enter second string: world
Not Anagrams
```

22. Remove Duplicates from String

Code:

```
import java.util.LinkedHashSet;
import java.util.Scanner;
public class RemoveDuplicates {
  public static void main(String[] args) {
     Scanner sc = new Scanner(System.in);
     System.out.print("Enter a string: ");
     String str = sc.nextLine();
     StringBuilder sb = new StringBuilder();
     LinkedHashSet<Character> set = new LinkedHashSet<>();
     for (char ch : str.toCharArray()) {
         set.add(ch);      // Set keeps only unique chars
     }
     for (char ch : set) {
         sb.append(ch);    // Preserve order
     }
     System.out.println("String after removing duplicates: "
                        + sb.toString());
     sc.close();
  }
}
```

Explanation:

- Reads a string.
- Uses LinkedHashSet to store unique characters (maintains insertion order).
- Builds a new string from the set.
- Prints the string without duplicates.

Output:

```
Enter a string: programming
String after removing duplicates: progamin
```

23. Find Second Largest Number

Code:

```
import java.util.Scanner;
public class SecondLargesk {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("How many numbers? ");
      int n = sc.nextInt();
      int largest = Integer.MIN_VALUE;
      int second = Integer.MIN_VALUE;
      for (int i = 0; i < n; i++) {
          System.out.print("Enter number: ");
          int num = sc.nextInt();
          if (num > largest) {
              second = largest;
              largest = num;
          } else if (num > second && num < largest) {
              second = num;
          }
      }
      if (second == Integer.MIN_VALUE)
          System.out.println("Second largest not found");
      else
          System.out.println("Second largest number: " + second);
      sc.close();
   }
}
```

Explanation:

- Reads n numbers.
- Keeps track of largest and second largest.
- Updates values while iterating.
- If second largest doesn't exist, prints a message.
- Otherwise prints it.

Output:

```
How many numbers? 5
Enter number: 12
Enter number: 45
Enter number: 23
Enter number: 67
Enter number: 34
Second largest number: 45
```

24. Sum of Natural Numbers

Code:

```
import java.util.Scanner;
public class SumNaturalNumbers {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a number: ");
      int n = sc.nextInt();
      long sum = 0;
      for (int i = 1; i <= n; i++) {
          sum += i;
      }
      System.out.println("Sum of first " + n
                  + " natural numbers: " + sum);
      sc.close();
   }
}
```

Explanation:

- Reads a number n.
- Adds all numbers from 1 to n.
- Uses long for large sums.
- Prints the total sum.

Output:

```
Enter a number: 100
Sum of first 100 natural numbers: 5050
```

Tip: Understand the logic, practice implementations, and try edge cases (empty input, large numbers, duplicates, etc.) to master problem solving!

25. Perfect Number Check

Code:

```
import java.util.Scanner;
public class PerfectNumber {
  public static void main(String[] args) {
     Scanner sc = new Scanner(System.in);
     System.out.print("Enter a number: ");
     int n = sc.nextInt();
     int sum = 0;
     for (int i = 1; i < n; i++) {
         if (n % i == 0) {
            sum += i;
         }
     }
     if (sum == n) {
         System.out.println(n + " is a
           Perfect Number.");
     } else {
         System.out.println(n + " is Not
           a Perfect Number.");
     }
     sc.close();
  }
}
```

Explanation:

- Reads a number n.
- Finds all divisors of n excluding n itself.
- Adds those divisors.
- If the sum equals n, it is a Perfect Number.
- Otherwise, it is not.

Output:

Example 1 (Perfect Number)

```
Enter a number: 28
28 is a Perfect Number.
```

Example 2 (Not Perfect Number)

```
Enter a number: 12
12 is Not a Perfect Number.
```

26. Decimal to Binary Conversion

Code:

```
import java.util.Scanner;
public class DecimalToBinary {
  public static void main(String[] args) {
     Scanner sc = new Scanner(System.in);
     System.out.print("Enter a decimal number: ");
     int n = sc.nextInt();
     if (n == 0) {
        System.out.println("Binary: 0");
     } else {
        StringBuilder bin = new StringBuilder();
        int temp = n;
        while (temp > 0) {
            int rem = temp % 2;
            bin.insert(0, rem);   // prepend remainder
            temp /= 2;
        }
        System.out.println("Binary: " + bin.toString());
     }
     sc.close();
  }
}
```

Explanation:

- Reads a decimal number n.
- Repeatedly divides n by 2 and stores the remainder (0 or 1).
- Remainders are collected in reverse order.
- Finally prints the binary representation.

Output:

Example 1

```
Enter a decimal number: 10
Binary: 1010
```

Example 2

```
Enter a decimal number: 25
Binary: 11001
```

27. Binary to Decimal Conversion

Code:

```
import java.util.Scanner;
public class BinaryToDecimal {
  public static void main(String[] args) {
     Scanner sc = new Scanner(System.in);
     System.out.print("Enter a binary number: ");
     String bin = sc.next();
     int len = bin.length();
     int decimal = 0;
     for(int i = 0; i < len; i++) {
         char ch = bin.charAt(i);
         if (ch == '1') {
            decimal += Math.pow(2, len - i - 1);
         } else if (ch != '0') {
            System.out.println("Invalid binary
                    " + ch);
            sc.close();
            return;
         }
     }
     System.out.println("Decimal: " + decimal);
     sc.close();
  }
}
```

Explanation:

- Reads a binary number as a string.
- For each bit (1 or 0) from left to right:
  - If bit is 1, add $2^{position}$.
- Sum of all such values is the decimal equivalent.
- Also validates that all characters are 0 or 1.

Output:

Example 1

```
Enter a binary number: 1011
Decimal: 11
```

Example 2

```
Enter a binary number: 11110
Decimal: 30
```

Example 3 (Invalid Input)

```
Enter a binary number: 1021
Invalid binary digit: 2
```

Tip: Practice regularly, test with different inputs, and try to solve without looking at the solution. Happy Coding!

28. Check Leap Year

Code:

```java
import java.util.Scanner;
public class LeapYear {
   public static void main(String[] args) {
       Scanner sc = new Scanner(System.in);
       System.out.print("Enter a year: ");
       int year = sc.nextInt();
       boolean isLeap = false;

       if (year % 400 == 0) {
           isLeap = true;
       } else if (year % 100 == 0) {
           isLeap = false;
       } else if (year % 4 == 0) {
           isLeap = true;
       }

       if (isLeap)
           System.out.println(year + " is a Leap Year.");
       else
           System.out.println(year + " is Not a Leap Year.");

       sc.close();
   }
}
```

Explanation:

- Reads a year.
- A year is a leap year if:
  - Divisible by 400, or
  - Divisible by 4 but not by 100.
- Otherwise, it is not a leap year.
- Prints the result.

Output:

Example 1

```
Enter a year: 2024
2024 is a Leap Year.
```

Example 2

```
Enter a year: 2023
2023 is Not a Leap Year.
```

Example 3

```
Enter a year: 2000
2000 is a Leap Year.
```

29. Quadratic Equation Roots

Code:

```java
import java.util.Scanner;
public class QuadraticRoots {
  public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter coefficients a, b and c: ");
      double a = sc.nextDouble();
      double b = sc.nextDouble();
      double c = sc.nextDouble();

      double discriminant = b * b - 4 * a * c;

      if (discriminant > 0) {
          double r1 = (-b + Math.sqrt(discriminant)) / (2 * a);
          double r2 = (-b - Math.sqrt(discriminant)) / (2 * a);
          System.out.println("Roots are real and different:");
          System.out.printf("r1 = %.2f, r2 = %.2f", r1, r2);
      } else if (discriminant == 0) {
          double r = -b / (2 * a);
          System.out.println("Roots are real and equal:");
          System.out.printf("r1 = r2 = %.2f", r);
      } else {
          double real = -b / (2 * a);
          double imag = Math.sqrt(-discriminant) / (2 * a);
          System.out.println("Roots are complex:");
          System.out.printf("r1 = %.2f + %.2fi\n", real, imag);
          System.out.printf("r2 = %.2f - %.2fi", real, imag);
      }
      sc.close();
  }
}
```

Explanation:

- Solves quadratic equation $ax^2 + bx + c = 0$.
- Calculates discriminant $D = b^2 - 4ac$.
- If $D > 0$ → two distinct real roots.
- If $D = 0$ → two equal real roots.
- If $D < 0$ → two complex roots.
- Prints the roots accordingly.

Output:

Example 1 (Two real roots)

```
Enter coefficients a, b and c: 1 -3 2
Roots are real and different:
r1 = 2.00, r2 = 1.00
```

Example 2 (Equal roots)

```
Enter coefficients a, b and c: 1 2 1
Roots are real and equal:
r1 = r2 = -1.00
```

Example 3 (Complex roots)

```
Enter coefficients a, b and c: 1 2 5
Roots are complex:
r1 = -1.00 + 2.00i
r2 = -1.00 - 2.00i
```

30. Simple Interest Calculator

Code:

```java
import java.util.Scanner;
public class SimpleInterest {
  public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter Principal Amount (P): ");
      double P = sc.nextDouble();
      System.out.print("Enter Rate of Interest (R) in %: ");
      double R = sc.nextDouble();
      System.out.print("Enter Time (T) in years: ");
      double T = sc.nextDouble();

      double SI = (P * R * T) / 100;
      double Amount = P + SI;

      System.out.println("\n--- Simple Interest Result ---");
      System.out.printf("Principal Amount (P): %.2f\n", P);
      System.out.printf("Rate of Interest (R): %.2f%%\n", R);
      System.out.printf("Time (T): %.2f years\n", T);
      System.out.printf("Simple Interest (SI): %.2f\n", SI);
      System.out.printf("Total Amount (P + SI): %.2f\n", Amount);

     sc.close();
  }
}
```

Explanation:

- Reads Principal (P), Rate (R) and Time (T).
- Uses formula: $SI = (P \times R \times T) / 100$
- Total Amount = P + SI
- Displays all values clearly.

Output:

```
Enter Principal Amount (P): 10000
Enter Rate of Interest (R) in %: 8
Enter Time (T) in years: 2

--- Simple Interest Result ---
Principal Amount (P): 10000.00
Rate of Interest (R): 8.00%
Time (T): 2.00 years
Simple Interest (SI): 1600.00
Total Amount (P + SI): 11600.00
```

Tip: Understand the logic. Practice regularly and try different inputs to become a better programmer!

31. Square Star Pattern

Code:

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        System.out.print("*");
    }
    System.out.println();
}
```

Explanation:

- Prints * in n rows and n columns.
- Forms a solid square.

Output:

(n = 5)

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

32. Right Triangle Pattern

Code:

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        System.out.print("*");
    }
    System.out.println();
}
```

Explanation:

- Row i has i stars.
- Forms a right-angled triangle.

Output:

(n = 5)

```
*
* *
* * *
* * * *
* * * * *
```

33. Left Triangle Pattern

Code:

```
for (int i = n; i >= 1; i--) {
    for (int j = 1; j <= i; j++) {
        System.out.print("*");
    }
    System.out.println();
}
```

Explanation:

- Row n has n stars and decreases by 1.
- Forms a left triangle.

Output:

(n = 5)

```
* * * * *
* * * *
* * *
* *
*
```

34. Pyramid Pattern

Code:

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n - i; j++)
        System.out.print(" ");
    for (int j = 1; j <= 2*i-1; j++)
        System.out.print("*");
    System.out.println();
}
```

Explanation:

- Prints spaces then 2*i-1 stars.
- Forms a pyramid.

Output:

(n = 5)

```
    *
   ***
  *****
 *******
*********
```

35. Inverted Pyramid

Code:

```
for (int i = n; i >= 1; i--) {
    for (int j = 1; j <= n - i; j++)
        System.out.print(" ");
    for (int j = 1; j <= 2*i-1; j++)
        System.out.print("*");
    System.out.println();
}
```

Explanation:

- Starts from n and decreases.
- Forms inverted pyramid.

Output:

(n = 5)

```
*********
 *******
  *****
   ***
    *
```

36. Diamond Pattern

Code:

```
// upper part
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n - i; j++)
        System.out.print(" ");
    for (int j = 1; j <= 2*i-1; j++)
        System.out.print("*");
    System.out.println();
}
// lower part
// lower part
for (int i = n-1; i >= 1; i--) {
    for (int j = 1; j <= n - i; j++)
        System.out.print(" ");
    for (int j = 1; j <= 2*i-1; j++)
        System.out.print("*");
    System.out.println();
}
```

Explanation:

- Upper pyramid + inverted pyramid.
- Forms a diamond.

Output:

(n = 4)

37. Hollow Square Pattern

Code:

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        if (i == 1 || i == n ||
            j == 1 || j == n)
            System.out.print("*");
        else System.out.print(" ");
    }
    System.out.println();
}
```

Explanation:

- Prints * on borders and space inside.

Output:

(n = 5)

```
* * * * *
*       *
*       *
*       *
* * * * *
```

38. Hollow Pyramid

Code:

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n - i; j++)
        System.out.print(" ");
    for (int j = 1, j <= 2*i-1, j++) {
    if(j == 1 || j == 2*i-1 || i == n)
        System.out.print("*");
    else System.out.print(" ");
    }
    System.out.println();
}
```

Explanation:

- Hollow pyramid with stars on edges.

Output:

(n = 5)

```
    *
   * *
  *   *
 *     *
*********
```

39. Butterfly Pattern

Code:

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++)
        System.out.print("*");
    for (int j = 1; j <= 2*n-i); j++)
        System.out.print(" ");
    for (int j = 1; j <= i; j++)
        System.out.print("*");
    System.out.println();
}
for (int i = n; i >= 1; i--) {
    for (int j = 1; j <= i; j++)
        System.out.print("*");
    for (int j = 1; j <= 2*n-i); j++)
        System.out.print(" ");
    for (int j = 1, j <= i; j++)
        System.out.print("*");
    System.out.println();
}
```

Explanation:

- Two half pyramids joined in the middle.

Output:

(n = 4)

```
*      *
**    **
***  ***
********
**    **
```

40. Pascal Triangle

Code:

```
for (int i = 0; i < n; i++) {
    int num = 1;
    for (int j = 0; j <= i; j++) {
        System.out.print(num + " ");
        num = num * (i - j) / (j + 1);
    }
    System.out.println();
}
```

Explanation:

- Each number = nCr.
- Forms Pascal triangle.

Output:

(n = 5)

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

41. Number Triangle

Code:

```
int num = 1;
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        Syatem.out.print(num + " ");
        num++;
    }
    System.out.println();
}
```

Explanation:

- Prints numbers in increasing order.

Output:

(n = 4)

```
1
2 3
4 5 6
7 8 9 10
```

42. Floyd Triangle

Code:

```
int num = 1;
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print(num + " ");
        num++;
    }
    System.out.println();
}
```

Explanation:

- Similar to number triangle (Floyd's).

Output:

(n = 4)

```
1
2 3
4 5 6
7 8 9 10
```

43. Binary Triangle

Code:

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        if ((i + j) % 2 == 0)
            System.out.print("1 ");
        else System.out.print("0 ");
    }
    System.out.println();
}
```

Explanation:

- Prints 1 and 0 in alternate positions.

Output:

(n = 4)

```
1
0 1
1 0 1
0 1 0 1
```

44. Character Triangle

Code:

```
char ch = 'A';
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print(ch + " ");
        ch++;
    }
    System.out.println();
}
```

Explanation:

- Prints characters in increasing order.

Output:

(n = 4)

```
A
B C
D E F
G H I J
```

45. Sandglass Pattern

Code:

```
// upper
for (int i = n; i >= 1; i--) {
    for (int j = 1; j <= n - i; j++)
        System.out.print(" ");
    for (int j = 1; j <= 2*i-1; j++)
        System.out.print("*");
    System.out.println();
}
// lower
for (int i = 2; i <= n; i++) {
    for (int j = 1; j <= n - i; j++)
        System.out.print(" ");
    for (int j = 1, j <= 2*i-1, j++)
        System.out.print("*");
    System.out.println();
}
```

Explanation:

- Two pyramids joined at middle.

Output:

(n = 5)

```
*********
 *******
  *****
   ***
    *
   ***
  *****
 *******
*********
```

46. Hourglass Pattern

Code:

```
// upper
for (int i = n; i >= 1; i--) {
    for (int j = 1; j <= n - i; j++)
        System.out.print(" ");
    for (int j = 1; j <= 2*i-1; j++)
        System.out.print("*");
    System.out.println();
}

// lower
for (int i = 2; i <= n; i++) {
    for (int j = 1; j <= n - i; j++)
        System.out.print(" ");
    for (int j = 1, j <= 2*i-1, j++)
        System.out.print("*");
    System.out.println();
}
```

47. X Pattern

Code:

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        if (i == j || i + j == n + 1)
            System.out.print("*");
        else System.out.print(" ");
    }
    System.out.println();
}
```

Explanation:

- Stars on both diagonals.

Output:

(n = 5)

```
*     *
 *   *
  * *
 *   *
*     *
```

48. Zig-Zag Pattern

Code:

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        if ((i + j) % 2 == 0)
            System.out.print("1 ");
        else System.out.print("0 ");
    }
    System.out.println();
}
```

Explanation:

- Alternating 1 and 0 in zig-zag form.

Output:

(n = 4)

```
1 0 1 0
0 1 0 1
1 0 1 0
0 1 0 1
```

49. Hollow Diamond

Code:

```
// upper
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n - i; j++) {
        System.out.print(" ");
    for (int j = 1; j <= 2*i-1; j++) {
        if (j == 1 || j == 2*i-1 || i == n)
            System.out.print("*");
        else System.out.print(" ");
    }
    System.out.println();
}
// lower
for (int i = n-1; i >= 1; i--) {
    for (int j = 1; j <= n - i; j++) {
        System.out.print(" ");
    for (int j = 1; j <= 2*i-1; j++) {
        if (j == 1 || j == 2*i-1 || i == n)
            System.out.print("*");
        else System.out.print(" ");
    }
    System.out.println();
}
```

Explanation:

- Diamond with hollow inside.

Output:

(n = 4)

50. Rhombus Pattern

Code:

```
for (int i = 1; i <= n; i++) {
    for (int j = 1, j <= n - i; j++)
        System.out.print(" ");
        for (int j = 1; j <= n; j++)
    } *ezee;
    System.out.println();
}
```

Explanation:

- Spaces decrease, stars constant.

Output:

(n = 5)

51. Reverse Number Triangle

Code:

```
for (int i = 1; i <= n; i++) {
    for (int j = i; j >= 1; j--)
        System.out.print(j + " ");
    System.out.println();
}
```

Explanation:

- Numbers in reverse order per row.

Output:

(n = 4)

```
1
2 1
3 2 1
4 3 2 1
```

57. Staircase Pattern

Code:

```
int num = 2;
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        while (!isPrime(num)) num++;
        System.out.print(num + " ");
        num++;
    }
    System.out.println();
}
```

Explanation:

- Fills pyramid with prime numbers.

Output:

(n = 4)

```
2
3 5
7 11 13
17 19 23 29
```

53. Palinfoome Pyramid

Code:

```
for (int i = 1; i <= n; i++) {
    int num = 1;
    for (int j = 1; j <= i; j++)
        System.out.print(num++);
    for (int j = i-1; j >= 1; j--)
        System.out.print(--num);
    System.out.println();
}
```

Explanation:

- Numbers increase then decrease.

Output:

(n = 4)

```
1
1 2 1
1 2 3 2 1
1 2 3 4 3 2 1
```

54. Alphabet Pyramid

Code:

```
char ch = 'A';
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print(ch + " ");
    j  ch++;
    System.out.println();
}
```

Explanation:

- Alphabet pyramid in sequence.

Output:

(n = 4)

```
A
B C
D E F
G H I J
```

54. Checkoet Prattern

Code:

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        System.out.print((i-1)*n +
        + j + " "); = -- ");
    }
    System.out.println();
}
```

Explanation:

- Numbers printed in spiral layout.

Output:

(n = 4)

```
1   2   3   4
12  13  14  5
11  16  15  6
10  9   8   7
```

60. Border Stars Pattern

Code:

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        if (i == 1 || i == n ||
            j == 1 || j == n)
            System.out.print("*");
        else System.out.print(" ");
    }
    System.out.println();
}
```

Explanation:

- Hollow square with border stars.

(n = 4)

Output:

(n = 5)

```
*****
*   *
*   *
*   *
*****
```

Tip: Practice patterns regularly. Understand the logic, then try to code without looking!

31. Square Star Pattern

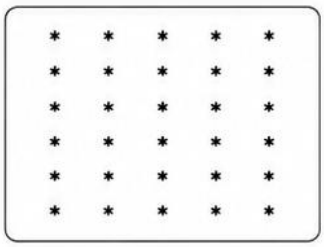
Code:

```
import java.util.*;
public class Pattern31 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      int n = sc.nextInt();
      for (int i = 1; i <= n; i++) {
         for (int j = 1; j <= n; j++) {
             System.out.print("* ");
         }
         System.out.println();
      }
   }
}
```

Explanation:

- Two nested loops run from 1 to n.
- Outer loop handles rows.
- Inner loop prints * n times.
- Forms a solid square.

Output (n = 5):



32. Right Triangle Pattern

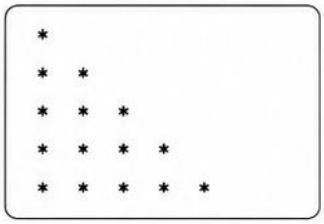
Code:

```
import java.util.*;
public class Pattern32 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      int n = sc.nextInt();
      for (int i = 1; i <= n; i++) {
         for (int j = 1; j <= i; j++) {
             System.out.print("* ");
         }
         System.out.println();
      }
   }
}
```

Explanation:

- For row i, inner loop runs from 1 to i.
- Number of stars increases each row.
- Forms a right-angled triangle.

Output (n = 5):



33. Left Triangle Pattern

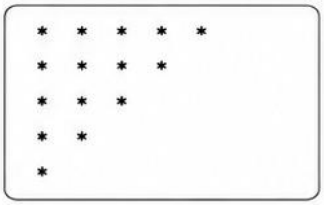
Code:

```
import java.util.*;
public class Pattern33 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      int n = sc.nextInt();
      for (int i = n; i >= 1; i--) {
         for (int j = 1; j <= i; j++) {
             System.out.print("* ");
         }
         System.out.println();
      }
   }
}
```

Explanation:

- Outer loop goes from n down to 1.
- Inner loop prints i stars.
- Number of stars decreases each row.
- Forms a left-angled triangle.

Output (n = 5):



34. Pyramid Pattern

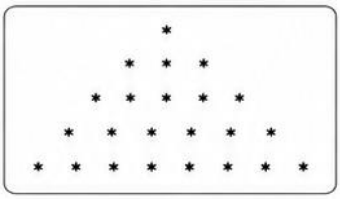
Code:

```
import java.util.*;
public class Pattern34 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      int n = sc.nextInt();
      for (int i = 1; i <= n; i++) {
         // spaces
         for (int j = 1; j <= n - i; j++) {
             System.out.print("  ");
         }
         // stars
         for (int j = 1; j <= 2 * i - 1; j++) {
             System.out.print("* ");
         }
         System.out.println();
      }
   }
}
```

Explanation:

- First loop prints leading spaces (n - i).
- Second loop prints 2*i - 1 stars.
- Stars increase by 2 each row.
- Forms a centered pyramid.

Output (n = 5):



35. Inverted Pyramid

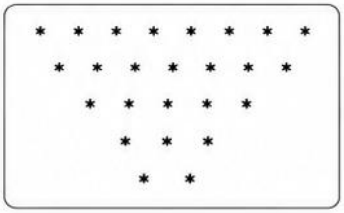
Code:

```
import java.util.*;
public class Pattern35 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      int n = sc.nextInt();
      for (int i = n; i >= 1; i--) {
         // spaces
         for (int j = 1; j <= n - i; j++) {
             System.out.print("  ");
         }
         // stars
         for (int j = 1; j <= 2 * i - 1; j++) {
             System.out.print("* ");
         }
         System.out.println();
      }
   }
}
```

Explanation:

- First loop prints leading spaces.
- Second loop prints 2*i - 1 stars.
- Stars decrease by 2 each row.
- Forms an inverted pyramid.

Output (n = 5):



36. Armstrong Number Check

Code:

```java
import java.util.Scanner;
public class Armstrong {
  public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a number: ");
      int n = sc.nextInt();
      int temp = n, sum = 0, digits = 0;
      int t = n;
      while (t > 0) {
          digits++;
          t /= 10;
      }
      while (temp > 0) {
          int d = temp % 10;
          sum += Math.pow(d, digits);
          temp /= 10;
      }
      if (sum == n)
        System.out.println(n + " is an Armstrong Number.");
      else
        System.out.println(n + " is Not an Armstrong Number.");
  }   sc.close();
}
```

Explanation:

- Reads a number n.
- Counts total digits in n.
- Raises each digit to the power of total digits and sums them.
- If sum equals n, it's an Armstrong number.
- Prints the result.

Output:

Example 1

```
Enter a number: 153
153 is an Armstrong Number.
```

Example 2

```
Enter a number: 123
123 is Not an Armstrong Number.
```

37. Strong Number Check

Code:

```java
import java.util.Scanner;
public class StrongNumber {
  public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a number: ");
      int n = sc.nextInt();
      int temp = n, sum = 0;
      while (temp > 0) {
          int d = temp % 10;
          int fact = 1;
          for (int i = 1; i <= d; i++)
              fact *= i;
          sum  += fact;
          temp /= 10;
      }
      if (sum == n)
        System.out.println(n + " is a Strong Number.");
      else
        System.out.println(n + " is Not a Strong Number.");
       sc.close();
  }
}
```

Explanation:

- Reads a number n.
- Finds factorial of each digit.
- Adds all factorials.
- If sum equals n, it's a Strong number.
- Prints the result.

Output:

Example 1

```
Enter a number: 145
145 is a Strong Number.
```

Example 2

```
Enter a number: 123
123 is Not a Strong Number.
```

38. Palindrome Number Check

Code:

```java
import java.util.Scanner;
public class PalindromeNumber {
  public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a number: ");
      int n = sc.nextInt();
      int temp = n, rev = 0;
      while (temp > 0) {
          int d = temp % 10;
          rev = rev * 10 + d;
          temp /= 10;
      }
      if (n == rev)
        System.out.println(n + " is a Palindrome Number.");
      else
        System.out.println(n + " is Not a Palindrome Number.");
  }
}
```

Explanation:

- Reads a number n.
- Reverses the number.
- If original equals reversed, it's a Palindrome number.
- Prints the result.

Output:

Example 1

```
Enter a number: 1221
1221 is a Palindrome Number.
```

Example 2

```
Enter a number: 1234
1234 is Not a Palindrome Number.
```

39. Sum of Digits

Code:

```java
import java.util.Scanner;
public class SumOfDigits {
  public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a number: ");
      int n = sc.nextInt();
      int sum = 0;
      while (n > 0) {
          sum += n % 10;
          n /= 10;
      }
      System.out.println("Sum of digits: " + sum);
  }
}
```

Explanation:

- Reads a number n.
- Extracts each digit using n % 10.
- Adds to sum.
- Prints the total sum.

Output:

Example 1

```
Enter a number: 9876
Sum of digits: 30
```

Example 2

```
Enter a number: 2051
Sum of digits: 8
```

40. Reverse a Number

Code:

```java
import java.util.Scanner;
public class ReverseNumber {
  public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a number: ");
      int n = sc.nextInt();
      int rev = 0;
      while (n > 0) {
          int d = n % 10;
          rev = rev * 10 + d;
          n /= 10;
      }
      System.out.println("Reverse: " + rev);
  }
}
```

Explanation:

- Reads a number n.
- Extracts last digit and appends to rev.
- Repeats until n becomes 0.
- Prints the reversed number.

Output:

Example 1

```
Enter a number: 4567
Reverse: 7654
```

Example 2

```
Enter a number: 1200
Reverse: 21
```

Tip: Practice these programs regularly. Try different inputs and trace the logic on paper to improve your problem-solving skills!

41. Number Triangle

Code:

```
import java.util.Scanner;
public class NumberTriangle {
  public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter n: ");
      int n = sc.nextInt();
      for (int i = 1; i <= n; i++) {
          for (int j = 1; j <= i; j++) {
              System.out.print(j + " ");
          }
          System.out.println();
      }
      sc.close();
  }
}
```

Explanation:

- Outer loop runs from 1 to n for rows.
- Inner loop runs from 1 to i for columns.
- Prints numbers from 1 to i in each row.

Output:

(n = 4)

```
1
1 2
1 2 3
1 2 3 4
```

42. Floyd Triangle

Code:

```
import java.util.Scanner;
public class FloydTriangle {
  public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter n: ");
      int n = sc.nextInt();
      int num = 1;
      for (int i = 1; i <= n; i++) {
          for (int j = 1; j <= i; j++) {
              System.out.print(num + " ");
              num++;
          }
          System.out.println();
      }
      sc.close();
  }
}
```

Explanation:

- Uses a variable num starting from 1.
- Prints successive numbers in each row.
- Numbers increase continuously across rows.

Output:

(n = 4)

```
1
2 3
4 5 6
7 8 9 10
```

43. Binary Triangle

Code:

```
import java.util.Scanner;
public class BinaryTriangle {
  public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter n: ");
      int n = sc.nextInt();
      for (int i = 1; i <= n; i++) {
          for (int j = 1; j <= i; j++) {
              if ((i + j) % 2 == 0)
                  System.out.print("1 ");
              else
                  System.out.print("0 ");
          }
          System.out.println();
      }
      sc.close();
  }
}
```

Explanation:

- Checks (i + j) % 2.
- If even, prints 1.
- If odd, prints 0.
- Produces alternating 1s and 0s in a triangle.

Output:

(n = 4)

```
1
0 1
1 0 1
0 1 0 1
```

44. Character Triangle

Code:

```
import java.util.Scanner;
public class BharacterTriangle {
  public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter n: ");
      int n = sc.nextInt();
      char ch = 'A';
      for (int i = 1; i <= n; i++) {
          for (int j = 1; j <= i; j++) {
              System.out.print(ch + " ");
              ch++;
          }
          System.out.println();
      }
      sc.close();
  }
}
```

Explanation:

- Uses a character variable ch starting from 'A'.
- Prints characters in increasing order.
- Continues across rows without reset.

Output:

(n = 4)

```
A
B C
D E F
G H I J
```

45. Sandglass Pattern

Code:

```
import java.util.Scanner;
public class SandglassPattern {
  public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter n: ");
      int n = sc.nextInt();

      // Upper part
      for (int i = n; i >= 1; i--) {
          for (int j = 1; j <= n - i; j++)
              System.out.print("  ");
          for (int j = 1; j <= (2 * i - 1); j++)
              System.out.print("* ");
          System.out.println();
      }
      // Lower part
      for (int i = 2; i <= n; i++) {
          for (int j = 1; j <= n - i; j++)
              System.out.print("  ");
          for (int j = 1; j <= (2 * i - 1); j++)
              System.out.print("* ");
          System.out.println();
      }
      sc.close();
  }
}
```

Explanation:

- Upper part: Starts with 2n-1 stars and reduces. Spaces increase.
- Lower part: Stars increase again while spaces decrease.
- Forms a sandglass (hourglass) shape.

Output:

(n = 5)

```
* * * * *
 * * * *
  * * *
   *
  * * *
 * * * *
* * * * *
```

Tip: Understand the pattern logic first, then code. Practice makes perfect!

46. Hourglass Pattern

Code:

```
import java.util.Scanner;
public class HourglassPattern {
  public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter n: ");
      int n = sc.nextInt();
      // Upper part
      for (int i = n; i >= 1; i--) {
          for (int j = 1; j <= n - i; j++)
              System.out.print(" ");
          for (int j = 1; j <= 2 * i - 1; j++)
              System.out.print("*");
          System.out.println();
      }
      // Lower part
      for (int i = 2; i <= n; i++) {
          for (int j = 1; j <= n - i; j++)
              System.out.print(" ");
          for (int j = 1; j <= 2 * i - 1; j++)
              System.out.print("*");
          System.out.println();
      }
  }
}
```

Explanation:

- Upper part: Stars decrease, spaces increase.
- Lower part: Stars increase, spaces decrease.
- Forms an hourglass shape.

Output:

(n = 5)

```
*******
 *****
  ***
   *
  ***
 *****
*******
```

47. X Pattern

Code:

```
import java.util.Scanner;
public class XPattern {
  public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter n: ");
      int n = sc.nextInt();
      for (int i = 1; i <= n; i++) {
       f  for (int j = 1; j <= n; j++) {
          if (i == j || i + j == n + 1)
              System.out.print("*");
          else
              System.out.print(" ");
      }
      System.out.println();
  }
}
```

Explanation:

- Prints * when row index equals column index (i == j) or their sum equals n + 1.
- Forms an 'X' shape.

Output:

(n = 5)

```
*   *
 * *
  *
 * *
*   *
```

48. Zig-Zag Pattern

Code:

```
import java.util.Scanner;
public class ZigZagPattern {
  public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter n: ");
      int n = sc.nextInt();
      for (int i = 1; i <= n; i++) {
          if (i % 2 == 1) {
              for (int j = 1; j <= n; j++)
                  System.out.print((j % 2 == 1) ? "*" : " ");
          } else {
              for (int j = 1; j <= n; j++)
                  System.out.print((j % 2 == 0) ? "*" : " ");
          }
          System.out.println();
      }
  }
}
```

Explanation:

- Odd rows: stars at odd positions.
- Even rows: stars at even positions.
- Creates a zig-zag pattern.

Output:

(n = 5)

```
* * *
 * *
* * *
 * *
* * *
```

49. Hollow Diamond

Code:

```
import java.util.Scanner;
public class HollowDiamond {
  public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter n: ");
      int n = sc.nextInt();
      // Upper part
      for (int i = 1; i <= n; i++) {
          for (int j = 1; j <= n - i; j++)
              System.out.print(" ");
          for (int j = 1; j <= 2 * i - 1; j++)
              System.out.print(i == 1 || i == n || j == 1 || j == 2 * i - 1 ? "*" : " ");
          System.out.println();
      }
      // Lower part
      for (int i = n - 1; i >= 1; i--) {
          for (int j = 1; j <= n - i; j++)
              System.out.print(" ");
          for (int j = 1; j <= 2 * i - 1; j++)
              System.out.print(i == 1 || i == n || j == 1 || j == 2 * i - 1 ? "*" : " ");
          System.out.println();
      }
  }
}
```

Explanation:

- Prints only border stars, inner spaces remain blank.
- Upper and lower parts together form a diamond.

Output:

(n = 4)

```
   *
  * *
 *   *
*     *
 *   *
  * *
   *
```

50. Rhombus Pattern

Code:

```
import java.util.Scanner;
public class RhombusPattern {
  public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter n: ");
      int n = sc.nextInt();
      for (int i = 1; i <= n; i++) {
          for (int j = 1; j <= n - i; j++)
              System.out.print(" ");
          for (int j = 1; j <= n; j++)
              System.out.print("*");
          System.out.println();
      }
  }
}
```

Explanation:

- Each row has n stars.
- Leading spaces decrease each row.
- Forms a right-slanted rhombus.

Output:

(n = 5)

```
 ****
 ****
 ****
 ****
 ****
```

Tip: Understand the logic pattern first, then code it yourself.

51. Reverse Number Triangle

Code:

```
import java.util.Scanner;
public class ReverseNumberTriangle {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter n: ");
      int n = sc.nextInt();
      for (int i = n; i >= 1; i--) {
         for (int j = 1; j <= i; j++) {
            System.out.print(j + " ");
         }
         System.out.println();
      }
   }
}
```

Explanation:

- Starts from n down to 1.
- Each row prints numbers from 1 to i.
- Produces a reverse number triangle.

Output:

(n = 5)

```
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1
```

52. Prime Pyramid

Code:

```
import java.util.Scanner;
public class PrimePyramid {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter n: ");
      int n = sc.nextInt();
      int num = 2;
      for (int i = 1; i <= n; i++) {
         for (int j = 1; j <= i; j++) {
            while (!isPrime(num)) {
               num++;
            }
            System.out.print(num + " ");
            num++;
         }
         System.out.println();
      }
      static boolean isPrime(int x) {
         if (x < 2) return false;
         for (int i = 2; i * i <= x; i++) {
            if (x % i == 0) return false;
         }
         return true;
      }
   }
}
```

Explanation:

- Prints prime numbers in a pyramid form.
- Numbers are placed left to right, row by row.
- Uses isPrime() to check prime numbers.

Output:

(n = 5)

```
2
3 5
7 11 13
17 19 23 29
31 37 41 43 47
```

53. Palindrome Pyramid

Code:

```
import java.util.Scanner;
public class PalindromePyramid {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter n: ");
      int n = sc.nextInt();
      for (int i = 1; i <= n; i++) {
         // decreasing part
         for (int j = i; j >= 1; j--) {
            System.out.print(j + " ");
         }
         // increasing part
         for (int j = 2; j <= i; j++) {
            System.out.print(j + " ");
         }
         System.out.println();
      }
   }
}
```

Explanation:

- First half decreases from i to 1.
- Second half increases from 2 to i.
- Each row reads the same forward and backward (palindrome).

Output:

(n = 5)

```
      1
    2 1 2
  3 2 1 2 3
4 3 2 1 2 3 4
5 4 3 2 1 2 3 4 5
```

54. Alphabet Pyramid

Code:

```
import java.util.Scanner;
public class AllpnabetPyramid {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter n: ");
      int n = sc.nextInt();
      for (int i = 1; i <= n; i++) {
         char ch = 'A';
         for (int j = 1; j <= i; j++) {
            System.out.print(ch + " ");
            ch++;
         }
         System.out.println();
      }
   }
}
```

Explanation:

- Starts from 'A' in each row.
- Prints next consecutive characters.
- Forms an alphabet pyramid.

Output:

(n = 5)

```
A
A B
A B C
A B C D
A B C D E
```

55. Spiral Pattern

Code:

```
import java.util.Scanner;
public class SpiralPattern {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter n: ");
      int n = sc.nextInt();
      int top = 1, bottom = n, left = 1, right = n;
      int num = 1;
      int[][] a = new int[n][n];
      while (top <= bottom && left <= right) {
         // left to right
         for (int j = left; j <= right; j++) a[top][j] = num++;
         top++;
         // top to bottom
         for (int i = top; i <= bottom; i++) a[i][right] = num++;
         right--;
         // right to left
         if (top <= bottom) {
            for (int j = right; j >= left; j--) a[bottom][j] = num++;
            bottom--;
         }
         // bottom to top
         if (left <= right) {
            for (int i = bottom; i >= top; i--) a[i][left] = num++;
            left++;
      }
      // print matrix
      for (int i = 0; i < n; i++) {
         for (int j = 0; j < n; j++) System.out.print(a[i][j] + " ");
         System.out.println();
      }
   }
}
```

Explanation:

- Fills the matrix in spiral order.
- Moves: left → right → top → bottom → right → left...
- Prints the spiral matrix.

Output:

(n = 5)

```
1  2  3  4  5
16 17 18 19 6
15 24 25 20 7
14 23 22 21 8
13 12 11 10 9
```

Tip: Practice all patterns with different values of n to strengthen your logic and improve speed!

56. Matrix Spiral Pattern

Code:

```java
import java.util.Scanner;
public class MatrixSpiral {
  public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter n: ");
      int n = sc.nextInt();

      int top = 0, bottom = n - 1;
      int left = 0, right = n - 1;
      int num = 1;
      int[][] a = new int[n][n];

      while (top <= bottom && left <= right) {
          for (int j = left; j <= right; j++)
              a[top][j] = num++;
          top++;

          for (int i = top; i <= bottom; i++)
              a[i][right] = num++;
          right--;

          if (top <= bottom) {
              for (int j = right; j >= left; j--)
                  a[bottom][j] = num++;
              bottom--;
          }

          if (left <= right) {
              for (int i = bottom; i >= top; i--)
                  a[i][left] = num++;
              left++;
          }
      }

      for (int i = 0; i < n; i++) {
          for (int j = 0; j < n; j++)
              System.out.printf("%3d", a[i][j]);
          System.out.println();
      }
      sc.close();
  }
}
```

Explanation:

- We fill the matrix layer by layer in a spiral order.
- Four boundaries are used: top, bottom, left, right.
- Fill top row (left → right) then shrink top.
- Fill right column (top → bottom) then shrink right.
- Fill bottom row (right → left) then shrink bottom.
- Fill left column (bottom → top) then shrink left.
- Repeat until boundaries cross.
- Finally, print the matrix.

Output:

(n = 5)

```
  1   2   3   4   5
 16  17  18  19   6
 15  24  25  20   7
 14  23  22  21   8
 13  12  11  10   9
```

57. Staircase Pattern

Code:

```java
import java.util.Scanner;
public class Staircaase {
  public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter n: ");
      int n = sc.nextInt();

      for (int i = 1; i <= n; i++) {
          for (int j = 1; j <= i; j++)
              System.out.print(j + " ");
          System.out.println();
      }
      sc.close();
  }
}
```

Explanation:

- Outer loop runs from 1 to n (rows).
- Inner loop runs from 1 to i (columns).
- Prints numbers from 1 to i in each row.
- Each row starts from 1 again.

Output:

(n = 5)

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

58. Arrow Pattern

Code:

```java
import java.util.Scanner;
public class ArrowPattern {
  public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter n: ");
      int n = sc.nextInt();

      // upper part
      for (int i = 1; i <= n; i++) {
          // spaces
          for (int j = 1; j <= n - i; j++)
              System.out.print("  ");
          // stars
          for (int j = 1; j <= 2 * i - 1; j++)
              System.out.print("* ");
          System.out.println();
      }
      // lower part
      for (int i = n - 1; i >= 1; i--) {
          // spaces
          for (int j = 1; j <= n - i; j++)
              System.out.print("  ");
          // stars
          for (int j = 1; j <= 2 * i - 1; j++)
              System.out.print("* ");
          System.out.println();
      }
      sc.close();
  }
}
```

Explanation:

- Upper part:

  Spaces decrease.

  Stars increase (odd count).
- Lower part:

  Spaces increase.

  Stars decrease.
- Total rows = 2n - 1.
- Forms an arrow shape.

Output:

(n = 5)

```
        *
      * * *
    * * * * *
  * * * * * * *
* * * * * * * * *
  * * * * * * *
    * * * * *
      * * *
        *
```

Tip: Break the pattern into parts. Identify what changes in each row (spaces, numbers, or stars) and code step by step!

59. Butterfly Pattern

Code:

```
import java.util.Scanner;
public class ButterflyPattern {
  public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter n: ");
      int n = sc.nextInt();

      // upper part
      for (int i = 1; i <= n; i++) {
          // stars
          for (int j = 1; j <= i; j++)
              System.out.print("* ");
          // spaces
          for (int j = 1; j <= 2*(n - i); j++)
              System.out.print("  ");
          // stars
          for (int j = 1; j <= i; j++)
              System.out.print("* ");
          System.out.println();
      }
      // lower part
      for (int i = n - 1; i >= 1; i--) {
          // stars
          for (int j = 1; j <= i; j++)
              System.out.print("* ");
          // spaces
          for (int j = 1; j <= 2*(n - i); j++)
              System.out.print("  ");
          // stars
          for (int j = 1; j <= i; j++)
              System.out.print("* ");
          System.out.println();
      }
      sc.close();
  }
}
```

Explanation:

- The pattern has two parts: upper and lower.
- Upper part:
  - Left stars increase from 1 to n.
  - Spaces decrease from 2*(n-1) to 0.
  - Right stars same as left.
- Lower part:
  - Left stars decrease from n-1 to 1.
  - Spaces increase from 2 to 2*(n-1).
  - Right stars same as left.
- Forms a butterfly shape.

Output:

(n = 5)

```
*                 *
* *             * *
* * *         * * *
* * * *     * * * *
* * * * * * * * * *
* * * *     * * * *
* * *         * * *
* *             * *
*                 *
```

60. Number Diamond

Code:

```
import java.util.Scanner;
public class NumberDiamond {
  public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter n: ");
      int n = sc.nextInt();

      // upper part
      for (int i = 1; i <= n; i++) {
          // leading spaces
          for (int j = n - i; j >= 1; j--)
              System.out.print("  ");
          // numbers increasing
          for (int j = 1; j <= i; j++)
              System.out.print(j + " ");
          // numbers decreasing
          for (int j = i - 1; j >= 1; j--)
              System.out.print(j + " ");
          System.out.println();
      }
      // lower part
      for (int i = n - 1; i >= 1; i--) {
          // leading spaces
          for (int j = n - i; j >= 1; j--)
              System.out.print("  ");
          // numbers increasing
          for (int j = 1; j <= i; j++)
              System.out.print(j + " ");
          // numbers decreasing
          for (int j = i - 1; j >= 1; j--)
              System.out.print(j + " ");
          System.out.println();
      }
      sc.close();
  }
}
```

Explanation:

- The pattern has two parts: upper and lower.
- Upper part:
  - Leading spaces decrease from n-1 to 0.
  - Numbers increase from 1 to i, then decrease from i-1 to 1.
- Lower part:
  - Leading spaces increase from 1 to n-1.
  - Numbers follow the same increase-then-decrease logic.
- Forms a number diamond.

Output:

(n = 5)

```
        1
      1 2 1
    1 2 3 2 1
  1 2 3 4 3 2 1
1 2 3 4 5 4 3 2 1
  1 2 3 4 3 2 1
    1 2 3 2 1
      1 2 1
        1
```

Tip: Break each pattern into parts (upper/lower, left/right, spaces/stars) on paper first. Then translate the pattern logic into loops.

Java Practice Question Bank

Java

Learn
Build
Grow

SECTION 3 – ARRAYS PROBLEMS (61-90)

Master Arrays with Logic & Practice

61. Array Input Output

Explanation: Read n elements into an array and print them.

Code (Java):

```
import java.util.*;
public class Q61 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++)
            arr[i] = sc.nextInt();
        for (int x : arr)
            System.out.print(x + " ");
    }
}
```

Output (Example):

```
Input:
5
10 20 30 40 50
---------------------
Output:
10 20 30 40 50
```

62. Array Sum

Explanation: Calculate the sum of all elements in the array.

Code (Java):

```
import java.util.*;
public class Q62 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        int sum = 0;
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
            sum += arr[i];
        }
        System.out.println(sum);
    }
}
```

Output (Example):

```
Input:
5
1 2 3 4 5
---------------------
Output:
15
```

63. Largest Element in Array

Explanation: Find and print the largest element in the array.

Code (Java):

```
import java.util.*;
public class Q63 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        int max = Integer.MIN_VALUE;
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
            if (arr[i] > max) max = arr[i];
        }
        System.out.println(max);
    }
}
```

Output (Example):

```
Input:
6
4 8 2 10 6 3
---------------------
Output:
10
```

Java Practice Question Bank

Java

SECTION 3 – ARRAYS PROBLEMS (61-90)

Learn Build Grow

64. Smallest Element in Array

Explanation: Find and print the smallest element in the array.

Logic:

- Assume first element as smallest.
- Traverse the array and update smallest if a smaller element is found.
- Print the smallest element.

Code (Java):

```
import java.util.*;
public class Q64 {
   public static void main(String[] args) {
       Scanner sc = new Scanner(System.in);
       int n = sc.nextInt();
       int[] arr = new int[n];
       for (int i = 0; i < n; i++)
           arr[i] = sc.nextInt();
       int min = arr[0];
       for (int i = 1; i < n; i++) {
           if (arr[i] < min) min = arr[i];
       }
       System.out.println(min);
   }
}
```

Output (Example):

```
Input:
6
7 3 9 2 5 8
--------------
Output:
2
```

65. Second Largest Element

Explanation: Find and print the second largest element in the array.

Logic:

- Find the largest element.
- Find the largest element smaller than the largest.
- Print the second largest element.

Code (Java):

```
import java.util.*;
public class Q65 {
   public static void main(String[] args) {
       Scanner sc = new Scanner(System.in);
       int n = sc.nextInt();
       int[] arr = new int[n];
       for (int i = 0; i < n; i++)
           arr[i] = sc.nextInt();
       int max = Integer.MIN_VALUE, smax = Integer.MIN_VALUE;
       for (int i = 0; i < n; i++) {
           if (arr[i] > max) {
               smax = max;
               max = arr[i];
           } else if (arr[i] > smax && arr[i] != max) {
               smax = arr[i];
           }
       }
       System.out.println(smax);
   }
}
```

Output (Example):

```
Input:
6
10 5 20 8 15 30
--------------
Output:
20
```

66. Reverse Array

Explanation: Reverse the array and print its elements.

Logic:

- Use two pointers: start and end.
- Swap elements at start and end.
- Move start forward and end backward until they meet.
- Print the reversed array.

Code (Java):

```
import java.util.*;
public class Q66 {
   public static void main(String[] args) {
       Scanner sc = new Scanner(System.in);
       int n = sc.nextInt();
       int[] arr = new int[n];
       for (int i = 0; i < n; i++)
           arr[i] = sc.nextInt();
       int start = 0, end = n - 1;
       while (start < end) {
           int temp = arr[start];
           arr[start] = arr[end];
           arr[end] = temp;
           start++;
           end--;
       }
       for (int i = 0; i < n; i++)
           System.out.print(arr[i] + " ");
   }
}
```

Output (Example):

```
Input:
5
1 2 3 4 5
-------------
Output:
5 4 3 2 1
```

Java Practice Question Bank

SECTION 3 – ARRAYS PROBLEMS (61-90)

Learn Build Grow

67. Sort Array Ascending

Explanation: Sort the array elements in ascending order and print them.

Logic:

- Read n elements of the array.
- Sort the array using Arrays.sort().
- Print the sorted array.

Code (Java):

```
import java.util.*;
public class Q67 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++)
            arr[i] = sc.nextInt();

        Arrays.sort(arr);
        for (int i = 0; i < n; i++)
            System.out.print(arr[i] + " ");
    }
}
```

Output (Example):

```
Input:
5
5 1 4 2 3
------------------
Output:
1 2 3 4 5
```

68. Sort Array Descending

Explanation: Sort the array elements in descending order and print them.

Logic:

- Read n elements of the array.
- Sort the array in ascending order.
- Print the array from last index to first index.

Code (Java):

```
import java.util.*;
public class Q68 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++)
            arr[i] = sc.nextInt();

        Arrays.sort(arr);
        for (int i = n - 1; i >= 0; i--)
            System.out.print(arr[i] + " ");
    }
}
```

Output (Example):

```
Input:
5
5 1 4 2 3
------------------
Output:
5 4 3 2 1
```

TIPS:

- For ascending order → use Arrays.sort(arr);
- For descending order → sort first, then traverse from end to start.
- Always test with different inputs (sorted, reverse, duplicates, single element).

Java Practice Question Bank

SECTION 3 – ARRAYS PROBLEMS (61-90)

Learn Build Grow

69. Linear Search

Explanation: Search for a given element in the array.

Logic:

- Read n elements of the array.
- Read the element to be searched (key).
- Traverse the array and compare each element with key.
- If found, print its index.
- If not found, print -1.

Code (Java):

```java
import java.util.*;
public class Q69 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++)
            arr[i] = sc.nextInt();
        int key = sc.nextInt();
        int index = -1;
        for (int i = 0; i < n; i++) {
            if (arr[i] == key) {
                index = i;
                break;
            }
        }
        System.out.print(index);
    }
}
```

Output (Example):

```
Input:
5
3 8 6 2 9
2
-----------------
Output:
3
```

* Indexing starts from 0.

70. Binary Search

Explanation: Search for a given element in a sorted array using binary search.

Logic:

- Read n elements of the sorted array.
- Read the element to be searched (key).
- Use two pointers: low (start) and high (end).
- Find mid and compare arr[mid] with key.
- If equal, print mid.
- If key is smaller, search in left half.
- If key is larger, search in right half.
- If not found, print -1.

Code (Java):

```java
import java.util.*;
public class Q70 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++)
            arr[i] = sc.nextInt();
        int key = sc.nextInt();
        int low = 0, high = n - 1, mid, index = -1;
        while (low <= high) {
            mid = (low + high) / 2;
            if (arr[mid] == key) {
                index = mid;
                break;
            } else if (key  < arr[mid]) {
                high = mid - 1;
            } else {
                low = mid  + 1;
            }
        }
        System.out.print(index);
    }
}
```

Output (Example):

```
Input:
7
2 4 6 8 10 12 14
10
-----------------
Output:
4
```

* Array must be sorted for Binary Search.

Java Practice Question Bank

Java

SECTION 3 – ARRAYS PROBLEMS (61–90)

Learn Build Grow

71. Frequency of Elements

Explanation: Find the frequency of each element in the array, and print it.

Logic:

- Read n elements of the array.
- For each element, count how many times it appears.
- Use a nested loop to count occurrence.
- Print the element and its frequency.

Code (Java):

```java
import java.util.*;
public class Q71 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++)
            arr[i] = sc.nextInt();

        boolean[] visited = new boolean[n];

        for (int i = 0; i < n; i++) {
            if (visited[i])
                continue;
            int count = 1;
            for (int j = i + 1; j < n; j++) {
                if (arr[i] == arr[j]) {
                    count++;
                    visited[j] = true;
                }
            }
            System.out.println(arr[i] + " -> " + count);
        }
    }
}
```

Output (Example):

Input:

```
7
1 2 2 3 1 4 2
```

Output:

```
1 -> 2
2 -> 3
3 -> 1
4 -> 1
```

Note:

Elements are printed in the order of their first occurrence.

72. Remove Duplicates

Explanation: Remove duplicates from the array and print the array with only unique elements.

Logic:

- Create a new array to store unique elements.
- Traverse the original array.
- If the element is not already in the new array,
- add it to the new array.
- Finally, print the new array.

Code (Java):

```java
import java.util.*;
public class Q72 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++)
            arr[i] = sc.nextInt();

        int[] temp = new int[n];
        int k = 0;
        for (int i = 0; i < n; i++) {
            boolean found = false;
            for (int j = 0; j < k; j++) {
                if (arr[i] == temp[j]) {
                    found = true;
                    break;
                }
            }
            if (!found)
                temp[k++] = arr[i];
        }
        for (int i = 0; i < k; i++)
            System.out.print(temp[i] + " ");
    }
}
```

Output (Example):

Input:

```
8
4 2 4 1 2 3 1 5
```

Output:

```
4 2 1 3 5
```

Note:

* Order of first occurrence is maintained.
* Extra space is used to store unique elements.

Java Practice Question Bank

Java

SECTION 3 – ARRAYS PROBLEMS (61–90)

Learn Build Grow

73. Merge Two Arrays

Explanation: Merge two arrays into a single array. and print the merged array.

Logic:

- Read both arrays.
- Create a new array of size n1 + n2.
- Copy elements of first array.
- Copy elements of second array.
- Print the merged array.

Code (Java):

```java
import java.util.*;
public class Q73 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      int n1 = sc.nextInt();
      int[] arr1 = new int[n1];
      for (int i = 0; i < n; i++)
          arr1[i] = sc.nextInt();

      int n2 = sc.nextInt();
      int[] arr2 = new int[n2];
      for (int i = 0; i < n; i++)
          arr2[i] = sc.nextInt();

      int[] merged = new int[n1 + n2];
      int i = 0, j = 0, k = 0;
      while (i < n1) merged[k++] = arr1[i++];
      while (j < n2) merged[k++] = arr2[j++];

      for (int x : merged)
          System.out.print(x + " ");
   }
}
```

Output (Example):

```
Input:
5
1 2 3 4 5
4
6 7 8 9
----------------
Output:
1 2 3 4 5 6 7 8 9
```

Note:

* The order of elements is preserved: first array followed by second array.

74. Left Rotate Array

Explanation: Left rotate the array by d positions and print the resulting array.

Logic:

- Read n and rotation count d.
- Store first d elements.
- Shift remaining elements to the left.
- Place stored elements at the end.
- Print the rotated array.

Code (Java):

```java
import java.util.*;
public class Q74 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      int n = sc.nextInt();
      int[] arr = new int[n];
      for (int i = 0; i < n; i++)
          arr[i] = sc.nextInt();

      int d = sc.nextInt();
      d = d % n;  // in case d >= n

      int[] temp = new int[d];
      for (int i = 0; i < d; i++)
          temp[i] = arr[i];

      for (int i = d; i < n; i++)
          arr[i - d] = arr[i];

      for (int i = 0; i < d; i++)
          arr[n - d + i] = temp[i];

      for (int x : arr)
          System.out.print(x + " ");
   }
}
```

Output (Example):

```
Input:
5
1 2 3 4 5
2
----------------
Output:
3 4 5 1 2
```

Note:

* Left rotate by d means the first d elements come to the end.
* Using extra array (temp) to store first d elements.

75. Right Rotate Array

Explanation: Right rotate the array by d positions and print the resulting array.

Logic:

- Read n and rotation count d.
- Store first n-d elements.
- Shift remaining elements to the right.
- Place stored elements at the beginning.
- Print the rotated array.

Code (Java):

```java
import java.util.*;
public class Q75 {
   public static void main(String[] args) {
       Scanner sc = new Scanner(System.in);
       int n = sc.nextInt();
       int d = sc.nextInt();
       int[] arr = new int[n];
       for (int i = 0; i < n; i++)
           arr[i] = sc.nextInt();

       d = d % n;   // in case d > n
       int[] temp = new int[d];

       for (int i = n - d; i < n; i++)
           temp[i - (n -d)] = arr[i];

       for (int i = n - d - 1; i >= 0; i--)
           arr[i + d] = arr[i];

       for (int i = 0; i < d; i++)
           arr[i] = temp[i];

       for (int x : arr)
           System.out.print(x + " ");
   }
}
```

Output (Example):

Input:

5 2

1 2 3 4 5

Output:

4 5 1 2 3

Note:

- ★ Right rotate by d means the last d elements come to the front.
- ★ Using extra array (temp) to store last d elements.

76. Find Missing Number

Explanation: Find the missing number from an array containing numbers from 1 to n (one number is missing).

Logic:

- Read n (size of array).
- Read n-1 numbers.
- Calculate expected sum = n*(n+1)/2.
- Subtract the actual sum from expected sum.
- The result is the missing number.

Code (Java):

```java
import java.util.*;
public class Q76 {
   public static void main(String[] args) {
       Scanner sc = new Scanner(System.in);
       int n = sc.nextInt();    // total numbers (1 to n)
       long expected = (long)n * (n + 1) / 2;
       long actual = 0;
       for (int i = 0; i < n - 1; i++)
           actual +=  sc.nextInt();

       long missing = expected - actual;
       System.out.println(missing);
   }
}
```

Output (Example):

Input:

5

1 2 4 5

Output:

3

Note:

- ★ Use long for sum to avoid integer overflow for large n.

Java Practice Question Bank

SECTION 3 – ARRAYS PROBLEMS (61-90)

Master Arrays with Logic & Practice

77. Find Duplicate Elements

1. CODE (Java)

```java
import java.util.*;
public class Q77 {
   public static void main(String[] args) {
      int[] arr = {4, 3, 2, 7, 8, 2, 3, 1};
      HashSet<Integer> seen = new HashSet<>();
      HashSet<Integer> duplicate = new HashSet<>();

      for (int num : arr) {
          if (seen.contains(num)) {
              duplicate.add(num);
          } else {
              seen.add(num);
          }
      }
      System.out.println("Duplicate Elements: " + duplicate);
   }
}
```

2. CODE LOGIC EXPLANATION

- We use two HashSet:
  - seen → to store elements we have already seen.
  - duplicate → to store elements that appear more than once.
- Traverse the array:
  - If element is already in seen, add it to duplicate.
  - Otherwise, add it to seen.
- Finally, print all elements in duplicate set.

3. OUTPUT

```
Duplicate Elements: [2, 3]
```

78. Find Unique Elements

1. CODE (Java)

```java
import java.util.*;
public class Q78 {
   public static void main(String[] args) {
      int[] arr = {4, 3, 2, 7, 8, 2, 3, 1, 9};
      HashMap<Integer, Integer> map = new HashMap<>();

      // Count frequency of each element
      for (int num : arr) {
         map.put(num, map.getOrDefault(num, 0) + 1);
      }

      System.out.print("Unique Elements: ");
      for (Map.Entry<Integer, Integer> entry : map.entrySet()) {
         if (entry.getValue() == 1) {
            System.out.print(entry.getKey() + " ");
         }
      }
   }
}
```

2. CODE LOGIC EXPLANATION

- Use a HashMap to count frequency of each element.
- Traverse the array and update the count in the map.
- After traversal, elements with frequency 1 are unique.
- Print those elements.

3. OUTPUT

```
Unique Elements: 4 7 8 1 9
```

Java Practice Question Bank

Java

SECTION 3 – ARRAYS PROBLEMS (61-90)

Master Arrays with Logic & Practice

79. Pair with Given Sum

1. CODE (Java)

```
import java.util.*;
public class Q79 {
   public static void main(String[] args) {
     int[] arr = {1, 4, 5, 7, -1, 5};
     int target = 6;
     Set<Integer> seen = new HashSet<>();
     boolean found = false;

     for (int num : arr) {
         int needed = target - num;
         if (seen.contains(needed)) {
            System.out.println("Pair Found: " + needed
                        + " , " + num);
            found = true;
            break;
        }
        seen.add(num);
     }
     if (!found) {
        System.out.println("No Pair Found.");
     }
  }
}
```

2. CODE LOGIC EXPLANATION

- We use a HashSet to store elements we have seen.
- For each element, check if target - element exists in the set.
- If yes, a pair is found.
- Otherwise, add the current element to the set.
- If no pair is found till the end, print "No Pair Found."

3. OUTPUT

```
Pair Found: 1 , 5
```

80. Kadane's Algorithm

1. CODE (Java)

```
import java.util.*;
public class Q80 {
   public static void main(String[] args) {
     int[] arr = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
     int maxSoFar = arr[0];
     int maxEndingHere = arr[0];

     for (int i = 1; i < arr.length; i++) {
        maxEndingHere = Math.max(arr[i],
                          maxEndingHere + arr[i]);
        maxSoFar = Math.max(maxSoFar, maxEndingHere);
     }

   System.out.println("Maximum Subarray Sum: "+ maxSoFar);
  }
}
```

2. CODE LOGIC EXPLANATION

- maxEndingHere stores the maximum subarray sum ending at current index.
- At each step, decide whether to extend the previous subarray or start a new subarray from current element.
- maxSoFar keeps track of the overall maximum subarray sum.
- Finally, print maxSoFar.

3. OUTPUT

```
Maximum Subarray Sum: 6
```

Java Practice Question Bank

Java

SECTION 3 – ARRAYS PROBLEMS (61-90)

→ Master Arrays with Logic & Practice ←

81. Maximum Product Subarray

1. CODE (Java)

```java
import java.util.*;
public class Q81 {
  public static void main(String[] args) {
    int[] arr = {2, 3, -2, 4};
    int maxProd = maxProduct(arr);
    System.out.println("Maximum Product: " + maxProd);
 }

 public static int maxProduct(int[] arr) {
   int maxSoFar = arr[0];
   int maxEndingHere = arr[0];
   int minEndingHere = arr[0];
   for (int i = 1; i < arr.length; i++) {
       if (arr[i] < 0) {
          int temp = maxEndingHere;
          maxEndingHere = minEndingHere;
          minEndingHere = temp;
      }
      maxEndingHere = Math.max(arr[i], maxEndingHere * arr[i]);
      minEndingHere = Math.min(arr[i], minEndingHere * arr[i]);
      maxSoFar = Math.max(maxSoFar, maxEndingHere);
   }
   return maxSoFar;
 }
}
```

2. CODE LOGIC EXPLANATION

- The maximum product subarray can change sign when we encounter a negative number.
- We keep track of both maximum and minimum product ending at current index.
- If current number is negative, swap maxEndingHere and minEndingHere.
- Update both values by considering current element alone or by extending the previous product.
- Update the overall maximum product.

3. OUTPUT

```
Maximum Product: 6
```

82. Equilibrium Index

1. CODE (Java)

```java
import java.util.*;
public class Q82 {
  public static void main(String[] args) {
    int[] arr = {-7, 1, 5, 2, -4, 3, 0};
    int index = findEquilibriumIndex(arr);
    System.out.println("Equilibrium Index: " + index);
 }

 public static int findEquilibriumIndex(int[] arr) {
   int totalSum = 0;
   for (int num : arr) totalSum += num;

   int leftSum = 0;
   for (int i = 0; i < arr.length; i++) {
      totalSum -= arr[i]; // right sum
      if (leftSum == totalSum) return i;
      leftSum += arr[i];
   }
   return -1;  // no equilibrium index
 }
}
```

2. CODE LOGIC EXPLANATION

- First, calculate the total sum of the array.
- Traverse the array while maintaining a left sum.
- For each index `i`:
  - Subtract arr[i] from total sum to get right sum.
  - If left sum equals right sum, `i` is the equilibrium index.
  - Otherwise, add arr[i] to left sum.
- If no index satisfies the condition, return `-1`.

3. OUTPUT

```
Equilibrium Index: 3
```

Java Practice Question Bank

Java

SECTION 3 – ARRAYS PROBLEMS (61-90)

Master Arrays with Logic & Practice

83. Majority Element

1. CODE (Java)

```java
import java.util.*;
public class Q83 {
   public static int majorityElement(int[] arr) {
      int count = 1, candidate = arr[0];
      for (int i = 1; i < arr.length; i++) {
         if (arr[i] == candidate) {
            count++;
         } else {
            count--;
            if (count == 0) {
               candidate = arr[i];
               count = 1;
            }
         }
      }
      count = 0;
      for (int num : arr) {
         if (num == candidate) count++;
      }
      return (count > arr.length / 2) ? candidate : -1;
   }
   public static void main(String[] args) {
      int[] arr = {2, 2, 1, 1, 1, 2, 2};
      System.out.println("Majority Element: " +
                  majorityElement(arr));
   }
}
```

2. CODE LOGIC EXPLANATION

- Use Boyer-Moore Voting Algorithm.
- We select a candidate by pairing different elements.
- After first pass, candidate is potential majority element.
- Verify by counting its actual occurrences.
- If count > n/2, it is the majority element; otherwise, return -1.

3. OUTPUT

```
Majority Element: 2
```

84. Move Zeros to End

1. CODE (Java)

```java
import java.util.*;
public class Q84 {
   public static void moveZerosToEnd(int[] arr) {
      int j = -1;
      for (int i = 0; i < arr.length; i++) {
         if (arr[i] == 0) {
            j = i;
            break;
         }
      }
      if (j == -1) return; // no zero present
      for (int i = j + 1; i < arr.length; i++) {
         if (arr[i] != 0) {
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
            j++;
         }
      }
   public static void main(String[] args) {
      int[] arr = {0, 1, 0, 3, 12};
      moveZerosToEnd(arr);
      System.out.print("Array: ");
      for (int num : arr) System.out.print(num + " ");
   }
}
```

2. CODE LOGIC EXPLANATION

- Find the first zero and store its index in j.
- Traverse the array from j+1 to end.
- If a non-zero element is found, swap it with the element at index j.
- Increment j.
- All non-zero elements come forward and zeros are pushed to the end.
- Order of non-zero elements is maintained.

3. OUTPUT

```
Array: 1 3 12 0 0
```

Java Practice Question Bank

Java

SECTION 3 – ARRAYS PROBLEMS (61-90)

Master Arrays with Logic & Practice

85. Union of Two Arrays

1. CODE (Java)

```java
import java.util.*;
public class Q85 {
   public static void main(String[] args) {
      int[] arr1 = {7, 1, 5, 2, 2, 3};
      int[] arr2 = {3, 8, 6, 7, 9, 1};
      HashSet<Integer> set = new HashSet<>();

      for (int num : arr1) set.add(num);
      for (int num : arr2) set.add(num);

      ArrayList<Integer> list = new ArrayList<>(set);
      Collections.sort(list);

      System.out.print("Union: ");
      for (int num : list) System.out.print(num + " ");
   }
}
```

2. CODE LOGIC EXPLANATION

- Use a HashSet to store unique elements from both arrays.
- Add all elements of arr1 and arr2 to the set.
- Convert the set to a list.
- Sort the list to display elements in ascending order.
- Print all elements → this is the union of both arrays.

3. OUTPUT

```
Union: 1 2 3 5 6 7 8 9
```

86. Intersection of Two Arrays

1. CODE (Java)

```java
import java.util.*;
public class Q86 {
   public static void main(String[] args) {
      int[] arr1 = {4, 9, 1, 2, 9, 5};
      int[] arr2 = {9, 4, 2, 3, 9, 7};

      HashMap<Integer, Integer> map = new HashMap<>();
      ArrayList<Integer> result = new ArrayList<>();

      for (int num : arr1)
          map.put(num, map.getOrDefault(num, 0) + 1);

      for (int num : arr2) {
          if (map.containsKey(num) && map.get(num) > 0) {
              result.add(num);
              map.put(num, map.get(num) - 1);
          }
      }
      Collections.sort(result);
      System.out.print("Intersection: ");
      for (int num : result) System.out.print(num + " ");
   }
}
```

2. CODE LOGIC EXPLANATION

- Store frequencies of elements of arr1 in a HashMap.
- Traverse arr2.
- If an element exists in the map with frequency > 0, it is part of the intersection.
- Add it to result list and reduce its frequency in the map.
- Sort the result list to print intersection in ascending order.

3. OUTPUT

```
Intersection: 2 4 9 9
```

Java Practice Question Bank

SECTION 3 – ARRAYS PROBLEMS (61–90)

Master Arrays with Logic & Practice

87. Leaders in an Array

1. CODE (Java)

```
import java.util.*;
public class Q87 {
   public static void main(String[] args) {
      int[] arr = {16, 17, 4, 3, 5, 2};
      int n = arr.length;
      List<Inteqer> leaders = new ArrayList<>();

      int maxFromRight = arr[n - 1];
      leaders.add(maxFromRight);
      for (int i = n - 2; i >= 0; i--) {
          if (arr[i] > maxFromRight) {
              maxFromRight = arr[i];
              leaders.add(arr[i]);
          }
      }
      Collections.reverse(leaders);
      System.out.print("Leaders: ");
      for (int num : leaders)
          System.out.print(num + " ");
   }
}
```

2. CODE LOGIC EXPLANATION

- A leader is an element which is greater than all elements to its right.
- Start from the rightmost element.
- Keep track of the maximum from right.
- If current element is greater than max from right, it is a leader.
- Add it to the list.
- Finally, reverse the list to maintain original order.

3. OUTPUT

```
Leaders: 17 5 2
```

88. Second Largest Element

1. CODE (Java)

```
import java.util.*;
public class Q87 {
   public static void main(String[] args) {
      int[] arr = {12, 35, 1, 10, 34, 1};
      if (arr.length < 2) {
          System.out.println("No second largest element.");
          return;
      }
      int first = Integer.MIN_VALUE;
      int second = Integer.MIN_VALUE;
      for (int num : arr) {
          if (num > first) {
              second = first;
              first = num;
          } else if (num > second && num != first) {
              second = num;
          }
      }
      if (second == Integer.MIN_VALUE)
          System.out.println("No second largest element.");
      else
          System.out.println("Second Largest: " + second);
   }
}
```

2. CODE LOGIC EXPLANATION

- Maintain two variables:
  first (largest) and second (second largest).
- Traverse the array:
  - If current > first: update second = first and first = current.
  - Else if current > second and current != first: update second.
- After traversal, second holds the second largest element.
- If second is still MIN_VALUE, it means there is no second largest element.

3. OUTPUT

```
Second Largest: 34
```

Java Practice Question Bank

SECTION 3 – ARRAYS PROBLEMS (61–90)

Master Arrays with Logic & Practice

89. Remove Duplicates from Sorted Array

1. CODE (Java)

```java
import java.util.*;
public class Q89 {
    public static int removeDuplicates(int[] arr) {
        if (arr.length == 0) return 0;
        int j = 0;
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] != arr[j]) {
                j++;
                arr[j] = arr[i];
            }
        }
        return j + 1; // new length
    }
    public static void main(String[] args) {
        int[] arr = {1, 1, 2, 2, 2, 3, 4, 4, 5};
        int newLen = removeDuplicates(arr);
        System.out.print("After removing duplicates: ");
        for (int i = 0; i < newLen; i++)
            System.out.print(arr[i] + " ");
        System.out.println("\nNew Length: " + newLen);
    }
}
```

2. CODE LOGIC EXPLANATION

- The array is sorted.
- Use two pointers:
  - i scans the array.
  - j keeps the position of the last unique element.
- If arr[i] is different from arr[j], move j forward and copy arr[i] to arr[j].
- At the end, elements from index 0 to j are unique.
- Return j + 1 as the new length.
- Print elements up to new length.

3. OUTPUT

```
After removing duplicates: 1 2 3 4 5
New Length: 5
```

90. Rotate Array by K Positions

1. CODE (Java)

```java
import java.util.*;
public class Q90 {
    public static void rotate(int[] arr, int k) {
        int n = arr.length;
        k = k % n; // handle k > n
        reverse(arr, 0, n - 1);
        reverse(arr, 0, k - 1);
        reverse(arr, k, n - 1);
    }
    private static void reverse(int[] arr, int start, int end) {
        while (start < end) {
            int temp = arr[start];
            arr[start] = arr[end];
            arr[end] = temp;
            start++;
            end--;
        }
    }
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5, 6, 7};
        int k = 3;
        rotate(arr, k);
        System.out.print("Rotated Array: ");
        for (int num : arr) System.out.print(num + " ");
    }
}
```

2. CODE LOGIC EXPLANATION

- To rotate right by k positions:
  ① Reverse the entire array.
  ② Reverse the first k elements.
  ③ Reverse the remaining n – k elements.
- This brings last k elements to front and keeps order intact.
- Time Complexity: O(n)
- Space Complexity: O(1)

3. OUTPUT

```
Rotated Array: 5 6 7 1 2 3 4
```

87. MATRIX ADDITION

Matrix addition is the process of adding two matrices of the same dimensions. The sum matrix contains the element-wise sum of the corresponding elements.

CODE (Java)

```java
import java.util.*;

public class MatrixAddition {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter rows: ");
    int r = sc.nextInt();
    System.out.print("Enter cols: ");
    int c = sc.nextInt();
    int[][] A = new int[r][c];
    int[][] B = new int[r][c];
    int[][] C = new int[r][c];

    System.out.println("Enter elements of Matrix A:");
    for (int i = 0; i < r; i++)
       for (int j = 0; j < c; j++)
           A[i][j] = sc.nextInt();
    System.out.println("Enter elements of Matrix B:");
    for (int i = 0; i < r; i++)
       for (int j = 0; j < c; j++)
           B[i][j] = sc.nextInt();
    // Add matrices
    for (int i = 0; i < r; i++)
       for (int j = 0; j < c; j++)
           C[i][j] = A[i][j] + B[i][j];
    System.out.println("Sum Matrix (A + B):");
    for (int i = 0; i < r; i++) {
       for (int j = 0; j < c; j++)
           System.out.print(C[i][j] + " ");
       System.out.println();
    }
    sc.close();
  }
}
```

LOGIC

1. Read rows (r) and cols (c).
2. Read matrix A of size r x c.
3. Read matrix B of size r x c.
4. Create result matrix C of size r x c.
5. For each cell (i, j):

$$C[i][j] = A[i][j] + B[i][j]$$

6. Print the sum matrix C.

EXAMPLE

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \qquad B = \begin{bmatrix} 6 & 5 & 4 \\ 3 & 2 & 1 \end{bmatrix}$$

OUTPUT

```
Sum Matrix (A + B):
7 7 7
7 7 7
```

---

88. MATRIX MULTIPLICATION

Matrix multiplication is performed when the number of columns in the first matrix equals the number of rows in the second matrix. The result matrix is obtained by the multiplication rule.

CODE (Java)

```java
import java.util.*;
public class MatrixMultiplication {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter rows of first matrix: ");
    int r1 = sc.nextInt();
    System.out.print("Enter cols of first matrix: ");
    int c1 = sc.nextInt();
    System.out.print("Enter rows of second matrix: ");
    int r2 = sc.nextInt();
    System.out.print("Enter cols of second matrix: ");
    int c2 = sc.nextInt();
    if (c1 != r2) {
       System.out.println("Multiplication not possible");
       sc.close();
       return;
    }
    int[][] A = new int[r1][c1];
    int[][] B = new int[r2][c2];
    int[][] C = new int[r1][c2];
    System.out.println("Enter elements of first matrix:");
    for (int i = 0; i < r1; i++)
       for (int j = 0; j < c1; j++)
           A[i][j] = sc.nextInt();
    System.out.println("Enter elements of second matrix:");
    for (int i = 0; i < r2; i++)
       for (int j = 0; j < c2; j++)
           B[i][j] = sc.nextInt();
    // Multiply matrices
    for (int i = 0; i < r1; i++)
       for (int j = 0; ; j< c2; j++)
           for (int k = 0; k < c1; k++)
               C[i][j] += A[i][k] * B[k][j];
    System.out.println("Product Matrix (A x B):");
    for (int i = 0; i < r1; i++) {
       for (int j = 0; j < c2; j++)
           System.out.print(C[i][j] + " ");
       System.out.println();
    }
    sc.close();
  }
}
```

LOGIC

1. Read dimensions of first matrix (r1 x c1).
2. Read dimensions of second matrix (r2 x c2).
3. Check if c1 == r2 (required condition).
4. Read both matrices A and B.
5. Create result matrix C of size (r1 x c2).
6. For each cell (i, j) in C:

$$C[i][j] = \sum_{k=0}^{c1-1} A[i][k] * B[k][j]$$

7. Print C.

EXAMPLE

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \qquad B = \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix}$$

CALCULATION

$$C[0][0] = 1 \times 7 + 2 \times 9 + 3 \times 11 = 58$$
$$C[0][1] = 1 \times 8 + 2 \times 10 + 3 \times 12 = 64$$
$$C[1][0] = 4 \times 7 + 5 \times 9 + 6 \times 11 = 139$$
$$C[1][1] = 4 \times 8 + 5 \times 10 + 6 \times 12 = 154$$

OUTPUT

```
Product Matrix (A x B):
58 64
139 154
```

Practice Daily
Master Arrays !

89. MATRIX TRANSPOSE

Transpose of a matrix is obtained by interchanging its rows and columns.
If A is an r x c matrix, then its transpose A^T is a c x r matrix.

CODE (Java)

```
import java.util.*;
public class MatrixTranspose {
   public static void main(String[] args) {
     Scanner sc = new Scanner(System.in);
     System.out.print("Enter rows (r): ");
     int r = sc.nextInt();
     System.out.print("Enter cols (c): ");
     int c = sc.nextInt();
     int[][] A = new int[r][c];

     System.out.println("Enter elements of Matrix:");
     for (int i = 0; i < r; i++)
         for (int j = 0; j < c; j++)
             A[i][j] = sc.nextInt();

     // Transpose: A^T will be c x r matrix
     int[][] T = new int[c][r];
     for (int i = 0; i < r; i++)
         for (int j = 0; j < c; j++)
             T[j][i] = A[i][j];

     System.out.println("Transpose of Matrix (A^T):");
     for (int i = 0; i < c; i++) {
         for (int j = 0; j < r; j++)
             System.out.print(T[i][j] + " ");
         System.out.println();
     }
     sc.close();
 }
}
```

LOGIC

1. Read rows r and columns c.
2. Read matrix A of size r x c.
3. Create transpose matrix T of size c x r.
4. For each cell (i, j):

   `T[j][i] = A[i][j]`

5. Print T.

EXAMPLE

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5. & 6 \end{bmatrix} \qquad A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

OUTPUT

```
Transpose of Matrix (A^T):
1   4
2   5
3   6
```

90. SPIRAL MATRIX TRAVERSAL

Spiral Matrix Traversal is the process of visiting all elements of a matrix in spiral order (clockwise) starting from the top-left element.

CODE (Java)

```
import java.util.*;
public class SpiralMatrixTraversal {
   public static void main(String[] args) {
     Scanner sc = new Scanner(System.in);
     System.out.print("Enter rows (r): ");
     int r = sc.nextInt();
     System.out.print("Enter cols (c): ");
     int c = sc.nextInt();
     int[][] A = new int[r][c];

     System.out.println("Enter elements of Matrix:");
     for (int i = 0; i < r; i++)
         for (int j = 0; j < c; j++)
             A[i][j] = sc.nextInt();

     List<Integer> result = new ArrayList<>();

     int top = 0, bottom = r - 1, left = 0, right = c - 1;

     while (top <= bottom && left <= right) {
         // 1. Left to Right
         for (int j = left; j <= right; j++)
             result.add(A[top][j]);
         top++;
     // 2. Top to Bottom
         for (int i = top; i <= bottom; i++)
             result.add(A[i][right]);
         right--;
     // 3. Right to Left
     if (top <= bottom) {
         for (int j = right; j >= left; j--)
             result.add(A[bottom][j]);
         bottom--;
     }
     // 4. Bottom to Top
     if (left <= right) {
         for (int i = bottom; i >= top; i--)
             result.add(A[i][left]);
         left++;
     }

   System.out.println("Spiral Order: " + result);
   sc.close();
 }
}
```

LOGIC

1. Initialize four pointers:

   `top = 0, bottom = r - 1, left = 0, right = c - 1`

2. Traverse in four directions in order:
   - Left to Right → top row → top++
   - Top to Bottom → right column → right--
   - Right to Left → bottom row → bottom--
   - Bottom to Top → left column → left++
3. Repeat until top > bottom or left > right.
4. Print the elements in the order collected.

EXAMPLE

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix}$$

Spiral Order:

1, 2, 3, 4, 8, 12,
11, 10, 9, 5, 6, 7

OUTPUT

```
Spiral Order: [1, 2, 3, 4, 8, 12, 11, 10, 9, 5, 6, 7]
```

TIPS

- Always update pointers after each direction.
- Use while (top <= bottom && left <= right) to handle odd dimensions.
- Visualize the layers of the matrix.

Practice Daily, Master Arrays!

91. REVERSE A STRING

Reverse a string means to rearrange the characters of the string in the opposite order.

CODE (Java)

```java
import java.util.*;
public class ReverseString91 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a string: ");
      String str = sc.nextLine();
      String rev = "";
      // Traverse from end to start
      for (int i = str.length() - 1; i >= 0; i--) {
         rev = rev + str.charAt(i);
      }
      System.out.println("Reversed String: " + rev);
   }
}
```

LOGIC

1. Take the string as input.
2. Start from the last index of the string.
3. Append each character to a new string.
4. Print the reversed string.

EXAMPLE

Input: hello

Output: olleh

OUTPUT (Sample Run)

```
Enter a string: hello
Reversed String: olleh
```

---

92. PALINDROME STRING

A string is called a palindrome if it remains the same when reversed. (Ignoring case and spaces)

CODE (Java)

```java
import java.util.*;
public class PalindromeString92 {
  public static void main(String[] args) {
     Scanner sc = new Scanner(System.in);
     System.out.print("Enter a string: ");
     String str = sc.nextLine();
     // Remove spaces and convert to lowercase
     str = str.replaceAll("\\s+", "").toLowerCase();
     int left = 0, right = str.length() - 1;
     boolean isPalin = true;
     // Two pointer approach
     while (left < right) {
        if (str.charAt(left) != str.charAt(right)) {
           isPalin = false;
           break;
        }
        left++;
        right--;
     }
     if (isPalin)
        System.out.println("Palindrome String");
     else
        System.out.println("Not a Palindrome String");
  }
}
```

LOGIC

1. Take the string as input.
2. Remove spaces and convert to lowercase to ignore case and space.
3. Use two pointers (left and right).
4. Compare characters at both ends.
5. If all characters match, it is a palindrome.

EXAMPLE

Input: A man a plan a canal Panama

Output: Palindrome String

OUTPUT (Sample Run)

```
Enter a string: A man a plan a canal Panama
Palindrome String
```

TIPS

- Use StringBuilder for efficient reversal in large strings.
- For palindrome, ignore cases and spaces for accurate results.

- Two pointer approach is optimal for palindrome check (O(n) time).
- Practice more strings with mixed cases and spaces.

93. COUNT VOWELS

Count the number of vowels in the given string.

CODE (Java)

```java
import java.util.*;
public class CountVowels93 {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter a string: ");
    String str = sc.nextLine();

    // Count vowels
    int count = 0;
    str = str.toLowerCase();
    for (int i = 0; i < str.length(); i++) {
      char ch = str.charAt(i);
      if (ch=='a' || ch=='e' || ch=='i' ||
          ch=='o' || ch=='u') {
          count++;
      }
    }
    System.out.println("Number of Vowels: " + count);
  }
}
```

LOGIC

1. Take the string as input.
2. Convert the string to lowercase to handle both cases.
3. Traverse each character.
4. If the character is a, e, i, o, or u, increment the count.
5. Print the total count of vowels.

EXAMPLE

Input: Programming in Java

Output: 5

OUTPUT (Sample Run)

```
Enter a string: Programming in Java
Number of Vowels: 5
```

94. COUNT CONSONANTS

Count the number of consonants in the given string.

CODE (Java)

```java
import java.util.*;
public class CountConsonants94 {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter a string: ");
    String str = sc.nextLine();
    int count = 0;
    str = str.toLowerCase();

    // Count consonants
    for (int i = 0; i < str.length(); i++) {
      char ch = str.charAt(i);
      if (ch >= 'a' && ch <= 'z') {   // alphabet
        if (ch!='a' && ch!='e' &&
            ch!='i' && ch!='o' && ch!='u') {
            count++;
        }
      }
    }
    System.out.println("Number of Consonants: " + count);
  }
}
```

LOGIC

1. Take the string as input.
2. Convert the string to lowercase.
3. Traverse each character.
4. If the character is an alphabet (a-z) and not a vowel (a, e, i, o, u), then increment the consonant count.
5. Print the total count of consonants.

EXAMPLE

Input: Hello World

Output: 7

OUTPUT (Sample Run)

```
Enter a string: Hello World
Number of Consonants: 7
```

TIPS

- Use toLowerCase() for case-insensitive comparison.
- Vowels are: a, e, i, o, u
- Consonants are alphabetic characters excluding vowels.
- Ignore spaces, digits and special characters.
- Think before coding, code after understanding!

95. COUNT WORDS

Count the number of words in the given string.

CODE (Java)

```java
import java.util.*;
public class CountWords95 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a sentence: ");
      String str = sc.nextLine().trim();
      int count = 0;
      // If string is empty
      if (str.length() == 0) {
          System.out.println("Number of Words: 0");
          return;
      }
      // Split the string by one or more spaces
      String[] words = str.split("\\s+");
      count = words.length;
      System.out.println("Number of Words: " + count);
   }
}
```

LOGIC

1. Take the string as input.
2. Trim leading and trailing spaces.
3. If the string is empty, word count is 0.
4. Split the string using one or more spaces as delimiter.
5. The number of resulting parts is the number of words.
6. Print the count.

EXAMPLE

Input: Java is fun to learn

Output: 5

OUTPUT (Sample Run)

```
Enter a sentence: Java is fun to learn
Number of Words: 5
```

---

96. REMOVE SPACES

Remove all the spaces from the given string.

CODE (Java)

```java
import java.util.*;
public class RemoveSpaces96 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a string: ");
      String str = sc.nextLine();
      // Remove all spaces
      String result = str.replaceAll("\\s+", "");
      System.out.println("String after removing spaces: "
                         + result);
   }
}
```

LOGIC

1. Take the string as input.
2. Use replaceAll("\\s+", "") to replace one or more spaces with empty string.
3. The resulting string has no spaces.
4. Print the new string.

EXAMPLE

Input: a b c d

Output: abcd

OUTPUT (Sample Run)

```
Enter a string: a b c d
String after removing spaces: abcd
```

TIPS

- For counting words, split using "\\s+" to handle multiple spaces.
- For removing spaces, replaceAll is efficient and concise.
- Use trim() before splitting strings.
- Test with edge cases (empty string, only spaces, multiple spaces).

97. TOGGLE CASE

Toggle case means to convert uppercase letters to lowercase and lowercase letters to uppercase in the given string.

CODE (Java)

```java
import java.util.*;
public class ToggleCase97 {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter a string: ");
    String str = sc.nextLine();
    StringBuilder sb = new StringBuilder();

    // Toggle case of each character
    for (int i = 0; i < str.length(); i++) {
        char ch = str.charAt(i);
        if (Character.isUpperCase(ch)) {
            sb.append(Character.toLowerCase(ch));
        } else if (Character.isLowerCase(ch)) {
            sb.append(Character.toUpperCase(ch));
        } else {
            sb.append(ch); // Non-alphabetic
        }
    }

    System.out.println("Toggled String: " + sb.toString());
  }
}
```

LOGIC

1. Take the string as input.
2. Traverse each character.
3. If the character is uppercase, convert it to lowercase.
4. If the character is lowercase, convert it to uppercase.
5. If the character is not a letter, keep it unchanged.
6. Build the new string and print it.

EXAMPLE

Input: HeLLo WoRLD! 123

Output: hEllo wOrld! 123

OUTPUT (Sample Run)

```
Enter a string: HeLLo WoRLD! 123
Toggled String: hEllo wOrld! 123
```

---

98. UPPERCASE TO LOWERCASE

Convert all uppercase letters in the given string to lowercase.

CODE (Java)

```java
import java.util.*;
public class UppercaseToLowercase98 {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter a string: ");
    String str = sc.nextLine();
    StringBuilder sb = new StringBuilder();

    // Convert uppercase letters to lowercase
    for (int i = 0; i < str.length(); i++) {
        char ch = str.charAt(i);
        if (Character.isUpperCase(ch)) {
            sb.append(Character.toLowerCase(ch));
        } else {
            sb.append(ch); // Keep other characters same
    }
    System.out.println("Lowercase String: " + sb.toString());
  }
}
```

LOGIC

1. Take the string as input.
2. Traverse each character.
3. If the character is uppercase, convert it to lowercase.
4. If the character is not uppercase, keep it as it is.
5. Build the new string and print it.

EXAMPLE

Input: JAVA PROGRAMMING 101

Output: java programming 101

OUTPUT (Sample Run)

```
Enter a string: JAVA PROGRAMMING 101
Lowercase String: java programming 101
```

TIPS

- Use Character.isUpperCase(ch) and Character.isLowerCase(ch) to check the case.
- Use Character.toLowerCase(ch) and Character.toUpperCase(ch) to convert.
- StringBuilder is more efficient for building new strings.
- Always handle non-alphabetic characters carefully.

99. LOWERCASE TO UPPERCASE

Convert all lowercase letters in the given string to uppercase.

CODE (Java)

```java
import java.util.*;
public class LowerToUpper99 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a string: ");
      String str = sc.nextLine();
      StringBuilder sb = new StringBuilder();
      // Convert lowercase letters to uppercase
      for (int i = 0; i < str.length(); i++) {
          char ch = str.charAt(i);
          if (Character.isLowerCase(ch)) {
             sb.append(Character.toUpperCase(ch));
          } else {
             sb.append(ch);  // Keep other characters same
          }
      }
     }
    System.out.println("Uppercase String: " + sb.toString());
   }
}
```

LOGIC

1. Take the string as input.
2. Traverse each character.
3. If the character is lowercase, convert it to uppercase.
4. If the character is not lowercase, keep it unchanged.
5. Build the new string and print it.

EXAMPLE

Input: i love java

Output: I LOVE JAVA

OUTPUT (Sample Run)

```
Enter a string: i love java
Uppercase String: I LOVE JAVA
```

---

100. CHECK ANAGRAM STRINGS

Check whether two given strings are anagrams or not.
(Anagrams contain the same characters with the same frequency.)

CODE (Java)

```java
import java.util.*;
public class CheckAnagram100 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter first string: ");
      String s1 = sc.nextLine();
      System.out.print("Enter second string: ");
      String s2 = sc.nextLine();
      // Remove spaces and convert to lowercase
      s1 = s1.replaceAll("\\s+", "").toLowerCase();
      s2 = s2.replaceAll("\\s+", "").toLowerCase();
      // If lengths are different, not anagrams
      if (s1.length() != s2.length()) {
          System.out.println("Not Anagrams");
          return;
      }
      // Count frequency of characters
      int[] count = new int[256];
      for (int i = 0; i < s1.length(); i++) {
          count[s1.charAt(i)]++;
          count[s2.charAt(i)]--;
      }
      // If all frequencies are 0, they are anagrams
      for (int val : count) {
          if (val != 0) {
              System.out.println("Not Anagrams");
              return;
          }
      }
      System.out.println("Anagrams");
   }
}
```

LOGIC

1. Take two strings as input.
2. Remove spaces and convert both to lowercase (case-insensitive comparison).
3. If lengths are different, they cannot be anagrams.
4. Use a frequency array to count characters:
   - Increment for first string.
   - Decrement for second string.
5. If all frequencies are 0, strings are anagrams. Otherwise, not anagrams.

EXAMPLE

Input: listen
silent

Output: Anagrams

OUTPUT (Sample Run)

```
Enter first string: listen
Enter second string: silent
Anagrams
```

TIPS

- Always ignore spaces and case.
- Frequency array (size 256) works for extended ASCII characters.
- Time Complexity: O(n)
- Space Complexity: O(1)

101 Find Duplicate Characters

Question: Given a string, find and print all duplicate characters. Each duplicate character should be printed only once.

Example:

Input : programming
Output : r, p, g, m

Code (Java):

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        String str = "programming";
        str = str.toLowerCase();
        boolean[] seen = new boolean[256];
        boolean[] printed = new boolean[256];

        System.out.print("Duplicate characters: ");
        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            if (Character.isLetterOrDigit(ch)) {
                if (seen[ch] && !printed[ch]) {
                    System.out.print(ch + ", ");
                    printed[ch] = true;
                } else {
                    seen[ch] = true;
                }
            }
        }
    }
}
```

Logic:

1. Convert the string to lowercase to handle case insensitivity.
2. Use two boolean arrays of size 256 (for all ASCII characters).
   - seen[] → to mark first occurrence
   - printed[] → to track already printed duplicates
3. Traverse the string:
   - If character is letter or digit:
     - If seen before and not yet printed → print it and mark as printed.
     - Else mark it as seen.
4. This ensures each duplicate is printed only once.

Output:

```
Duplicate characters: r, p, g, m ,
```

102 Character Frequency

Question: Given a string, count the frequency of each character and print it.

Example:

Input : hello world
Output : h-1, e-1, l-3, o-2, (space)-1, w-1, r-1, d-1

Code (Java):

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        String str = "hello world";
        LinkedHashMap<Character, Integer> map =
                        new LinkedHashMap<>();
        for (char ch : str.toCharArray()) {
            map.put(ch, map.getOrDefault(ch, 0) + 1);
        }
        for (Map.Entry<Character, Integer> entry :
                        map.entrySet()) {
            char ch = entry.getKey();
            int count = entry.getValue();
            if (ch == '-') {
                System.out.print("(space)-" + count + ", ");
            } else {
                System.out.print(ch + "-" + count + ", ");
            }
        }
    }
}
```

Logic:

1. Use LinkedHashMap to store character as key and its frequency as value.
   - LinkedHashMap maintain's insertion order.
2. Traverse the string and for each character:
   - If already present → increment its count.
   - Else add with count = 1.
3. Traverse the map and print each character with its frequency.
4. If character is space, print as (space).

Output:

```
h-1, e-1, l-3, o-2, (space)-1,
w-1, r-1, d-1,
```

103 First Non-Repeating Character

Question: Given a string, find and print the first non-repeating character. If no such character exists, print -1.

Example:

Input : swiss
Output : w

Code (Java):

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String str = sc.nextLine();
        int[] freq = new int[256];
        for (char ch : str.toCharArray()) {
            freq[ch]++;
        }
        char ans = '\0';
        for (char ch : str.toCharArray()) {
            if (freq[ch] == 1) {
                ans = ch;
                break;
            }
        }
        if (ans == '\0')
            System.out.println(-1);
        else
            System.out.println(ans);
    }
}
```

Logic:

1. Create a frequency array of size 256 to store count of each character.
2. Traverse the string and update the frequency of each character.
3. Traverse the string again and return the first character whose frequency is 1 (non-repeating).
4. If no character has frequency 1, print -1.

Output:

Input 1 :	swiss	→	w
Input 2 :	aabbcc	→	-1
Input 3 :	teeter	→	r

---

104 First Repeating Character

Question: Given a string, find and print the first repeating character. If no such character exists, print -1.

Example:

Input : geeksforgeeks
Output : g

Code (Java):

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String str = sc.nextLine();
        int[] freq = new int[256];
        for (char ch : str.toCharArray()) {
            freq[ch]++;
            if (freq[ch] == 2) {
                System.out.println(ch);
                return;   // first repeating found
            }
        }
        System.out.println(-1);
    }
}
```

Logic:

1. Create a frequency array of size 256.
2. Traverse the string from left to right. For each character:
   - Increment its frequency.
   - If frequency becomes 2, it is the first repeating character → print it and stop.
3. If no character repeats, print -1.

Output:

Input 1 :	geeksforgeeks	→	g
Input 2 :	abcde	→	-1
Input 3 :	aabbccd	→	a

105 Remove Duplicate Characters

Question: Given a string, remove all duplicate characters and return the string with only distinct characters. Preserve the original order.

Example:

Input : programming
Output : progamin

Code (Java):

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        String str = "programming";
        boolean[] seen = new boolean[256];
        StringBuilder sb = new StringBuilder();
        for (char ch : str.toCharArray()) {
            if (!seen[ch]) {
                seen[ch] = true;
                sb.append(ch);
            }
        }
        System.out.println(sb.toString());
    }
}
```

Logic:

1. Create a boolean array of size 256 to mark seen characters.
2. Traverse the string.
   - If the character is not seen before, append it to result and mark it seen.
   - If already seen, skip it.
3. The result will contain only distinct characters in original order.

Output:

```
Input : programming
Output : progamin
```

106 Sort Characters in String

Question: Given a string, sort its characters in ascending alphabetical order.

Example:

Input : dcba
Output : abcd

Code (Java):

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        String str = "dcba";
        char[] arr = str.toCharArray();
        Arrays.sort(arr);
        String sorted = new String(arr);
        System.out.println(sorted);
    }
}
```

Logic:

1. Convert the string to a character array.
2. Sort the array using Arrays.sort().
3. Convert the sorted array back to string.
4. Print the sorted string.

Output:

```
Input : dcba  → abcd
Input : hello → ehllo
Input : zyx   → xyz
```

107 Reverse Words in Sentence

Question: Given a string sentence, reverse the order of words and return the reversed sentence. Words are separated by a single space.

Example:

Input : I love Java programming
Output : programming Java love I

Code (Java):

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String sentence = sc.nextLine().trim();
        // split by one or more spaces
        String[] words = sentence.split("\\s+");
        StringBuilder sb = new StringBuilder();
        for (int i = words.length - 1; i >= 0; i--) {
            sb.append(words[i]);
            if (i != 0) sb.append(" ");
        }
        System.out.println(sb.toString());
    }
}
```

Logic:

1. Read the sentence and split it into words using space as delimiter.
2. Traverse the array from the last word to the first.
3. Append each word to a StringBuilder with a space in between.
4. Print the final reversed sentence.

Output:

```
Input 1 :  I love Java programming
        → programming Java love I
Input 2 :  Hello World from Java
        → Java from World Hello
Input 3 :  One two three four
        → four three two One
```

108 Check Pangram

Question: Given a string, check if it is a pangram. A pangram is a sentence that contains every letter of the English alphabet at least once.

Example:

Input : The quick brown fox jumps over lazy dog
Output : true
Input : Hello World
Output : false

Code (Java):

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String str = sc.nextLine().toLowerCase();
        boolean[] seen = new boolean[26];
        int count = 0;
        for (char ch : str.toCharArray()) {
            if (ch >= 'a' && ch <= 'z') {
                int idx = ch - 'a';
                if (!seen[idx]) {
                    seen[idx] = true;
                    count++;
                }
            }
        }
        if (count == 26)
            System.out.println(true);
        else
            System.out.println(false);
    }
}
```

Logic:

1. Convert the string to lowercase.
2. Use a boolean array of size 26 to mark presence of each letter (a to z).
3. Traverse the string and mark letters seen in the array.
4. Count how many unique letters are seen.
5. If count is 26, it is a pangram, otherwise not.

Output:

```
Input 1 :  The quick brown fox jumps
           over lazy dog    → true
Input 2 :  Hello World      → false
Input 3 :  Sphinx of black quartz,
           judge my vow     → true
```

109 Remove Vowels

Question: Given a string, remove all the vowels (a, e, i, o, u – both lowercase and uppercase) and return the string.

Example:

Input : programming
Output : prgrmmng

Code (Java):

```
public class Main {
  public static void main(String[] args) {
     String str = "programming";
     StringBuilder sb = new StringBuilder();
     for (char ch : str.toCharArray()) {
         if (!isVowel(ch)) {
             sb.append(ch);
         }
     }
     System.out.print(sb.toString());
  }

  static boolean isVowel(char ch) {
     ch = Character.toLowerCase(ch);
     return (ch == 'a' || ch == 'e' ||
             ch == 'i' || ch == 'o' ||
             ch == 'u');
  }
}
```

Logic:

1. Traverse the string character by character.
2. For each character, check if it is a vowel (a, e, i, o, u – case insensitive).
3. If it is not a vowel, append it to StringBuilder.
4. After traversal, return the StringBuilder result as the final string.

Output:

Input 1 :	programming	→	prgrmmng
Input 2 :	Hello World	→	Hll Wrld
Input 3 :	Education	→	dctn

110 String Compression

Question: Given a string, compress it by replacing consecutive repeating characters with the character followed by the count.

Example:

Input : aabbbccaaa
Output : a2b3c2a3

Code (Java):

```
public class Main {
  public static void main(String[] args) {
     String str = "aabbbccaaa";
     StringBuilder sb = new StringBuilder();
     int count = 1;

     for (int i = 1; i < str.length(); i++) {
         if (str.charAt(i) == str.charAt(i - 1)) {
             count++;
         } else {
             sb.append(str.charAt(i - 1));
             sb.append(count);
             count = 1;
         }
     }
     // append last character and its count
     sb.append(str.charAt(str.length() - 1));
     sb.append(count);
     System.out.print(sb.toString());
  }
}
```

Logic:

1. Initialize count as 1.
2. Traverse the string from the second character.
3. If the current character is same as the previous one, increment count.
4. If different, append the previous character and its count to result, then reset count.
5. After the loop, append the last character and its count.

Output:

Input 1 :	aabbbccaaa	→	a2b3c2a3
Input 2 :	aaabccccddd	→	a3b1c4d3
Input 3 :	abc	→	a1b1c1

111 Run Length Encoding

Question : Given a string, perform Run Length Encoding. Replace consecutive repeating characters with the character followed by its count.

Example :

Input : aabbcccaaa
Output : a2b2c3a3

Code (Java):

```
public class Main {
public static String runLengthEncode(String str) {
    if (str == null || str.length() == 0) return "";
    StringBuilder sb = new StringBuilder();
    int count = 1;
    for (int i = 1; i < str.length(); i++) {
        if (str.charAt(i) == str.charAt(i - 1)) {
            count++;
        } else {
            sb.append(str.charAt(i - 1)).append(count);
            count = 1;
        }
    }
    // append last character and its count
    sb.append(str.charAt(str.length() - 1)).append(count);
    return sb.toString();
 }
public static void main(String[] args) {
     String str = "aabbcccaaa";
     System.out.println(runLengthEncode(str));
 }
}
```

Logic :

1. Initialize count of first character as 1.
2. Traverse the string from the second character.
3. If current character is same as previous, increment count.
4. If different, append previous character and count to result, then reset count.
5. After the loop, append the last character and its count.
6. Return the encoded string.

Output :

Input 1 :	aabbcccaaa	→	a2b2c3a3
Input 2 :	aaabbbcc	→	a3b3c2
Input 3 :	a	→	a1
Input 4 :	abcd	→	a1b1c1d1

112 Check Rotation of String

Question : Given two strings s1 and s2, check if s2 is a rotation of s1. (A string s2 is a rotation of s1 if it can be obtained by rotating s1 some number of times.)

Example :

Input : s1 = "waterbottle", s2 = "erbottlewat"
Output : true

Code (Java):

```
public class Main {
public static boolean isRotation(String s1, String s2) {
    if (s1 == null || s2 == null) return false;
    if (s1.length() != s2.length()) return false;
    if (s1.length() == 0) return true;
    String temp = s1 + s1;
    return temp.contains(s2);
 }
public static void main(String[] args) {
    String s1 = "waterbottle";
    String s2 = "erbottlewat";
    System.out.println(isRotation(s1, s2)); // true
  }
}
```

Logic :

1. If lengths of both strings are different, return false.
2. If both are empty, return true.
3. Concatenate s1 with itself.
4. If s2 is a substring of this concatenated string, then s2 is a rotation of s1.
5. Otherwise, return false.

Output :

Input 1 :	s1 = "waterbottle", s2 = "erbottlewat"	→	true
Input 2 :	s1 = "abcde", s2 = "cdeab"	→	true
Input 3 :	s1 = "abcde", s2 = "abced"	→	false
Input 4 :	s1 = "", s2 = ""	→	true

113 Check Subsequence

Question : Given two strings s1 and s2, check if s1 is a subsequence of s2. A string s1 is a subsequence of s2 if all characters of s1 appear in s2 in the same order (not necessarily contiguous).

Example :

Input : s1 = "abc", s2 = "ahbgdc"
Output : true

Code (Java):

```
public class Main {
 public static boolean isSubsequence(String s1, String s2) {
    int i = 0, j = 0;
    while (i < s1.length() && j < s2.length()) {
       if (s1.charAt(i) == s2.charAt(j)) {
          i++;
       }
       j++;
    }
    return i == s1.length();
 }

public static void main(String[] args) {
    String s1 = "abc";
    String s2 = "ahbgdc";
    System.out.println(isSubsequence(s1, s2)); // true
 }
}
```

Logic :

1. Traverse both strings using two pointers.
2. If characters match, move pointer in s1.
3. Always move pointer in s2.
4. If we reach the end of s1, it means all characters of s1 were found in order.
5. Return true if all characters of s1 are matched, else false.

Output :

```
Input 1 :  s1 = "abc", s2 = "ahbgdc"  → true
Input 2 :  s1 = "axc", s2 = "ahbgdc"  → false
Input 3 :  s1 = "", s2 = "abc"        → true
Input 4 :  s1 = "abc", s2 = ""        → false
```

114 Longest Common Prefix

Question : Given an array of strings, find the longest common prefix among all strings in the array. If there is no common prefix, return an empty string "".

Example :

Input : ["flower", "flow", "flight"]
Output : "fl"

Code (Java):

```
import java.util.*;
public class Main {
  public static String longestCommonPrefix(String[] strs) {
    if (strs == null || strs.length == 0) return "";
    String prefix = strs[0];
    for (int i = 1; i < strs.length; i++) {
       while (strs[i].indexOf(prefix) != 0) {
          prefix = prefix.substring(0, prefix.length() - 1);
          if (prefix.isEmpty()) return "";
       }
    }
    return prefix;
 }
public static void main(String[] args) {
    String[] strs1 = {"flower", "flow", "flight"};
    String[] strs2 = {"dog", "racecar", "car"};
    System.out.println(longestCommonPrefix(strs1)); // fl
    System.out.println(longestCommonPrefix(strs2)); // ""
 }
}
```

Logic :

1. Take the first string as initial prefix.
2. Compare this prefix with each string in the array.
3. Reduce the prefix character by character until all strings start with it.
4. If prefix becomes empty, return "".
5. After checking all strings, return the final prefix.

Output :

```
Input 1 :  ["flower", "flow", "flight"]      → "fl"
Input 2 :  ["dog", "racecar", "car"]         → ""
Input 3 :  ["interview", "internet", "internal"]
                                             → "inte"
Input 4 :  ["apple"]                         → "apple"
```

115 Longest Palindrome Substring

Question : Given a string s, return the longest substring which is a palindrome.

Example :

Input : babad
Output : bab (or aba)

Code (Java):

```
public class Main {
   public static String longestPalindrome(String s) {
      if (s == null || s.length() == 0) return "";
      int start = 0, maxLen = 1;
      for (int i = 0; i < s.length(); i++) {
         int len1 = expand(s, i, i);     // odd length
         int len2 = expand(s, i, i + 1); // even length
         int len = Math.max(len1, len2);
         if (len > maxLen) {
            maxLen = len;
            start = i - (len - 1) / 2;
         }
      }
      return s.substring(start, start + maxLen);
   }
   private static int expand(String s, int l, int r) {
      while (l >= 0 && r < s.length() &&
             s.charAt(l) == s.charAt(r)) {
         l--; r++;
      }
      return r - l - 1;
   }
   public static void main(String[] args) {
      String s = "babad";
      System.out.println(longestPalindrome(s)); // bab or aba
   }
}
```

Logic :

1. For every character, consider it as the center of a palindrome.
2. Expand around the center for odd length (i, i) and even length (i, i+1).
3. Calculate the length of the palindrome found in both cases.
4. Keep track of the maximum length and its starting index.
5. After checking all centers, return the substring with maximum length.

Output :

Input 1 :	babad	→	bab (or aba)
Input 2 :	cbbd	→	bb
Input 3 :	a	→	a
Input 4 :	forgeeksskeegfor	→	geeksskeeg

116 Longest Unique Substring

Question : Given a string s, find the length of the longest substring without repeating characters.

Example :

Input : abcabcbb
Output : 3 ("abc")

Code (Java):

```
public class Main {
   public static int lengthOfLongestSubstring(String s) {
      int n = s.length();
      if (n == 0) return 0;
      int[] index = new int[128];
      Arrays.fill(index, -1);
      int maxLen = 0, start = 0;
      for (int i = 0, i < n, i++) {
         char c = s.charAt(i);
         if (index[c] >= start) {
            start = index[c] + 1;
         }
         index[c] = i;
         maxLen = Math.max(maxLen, i - start + 1);
      }
      return maxLen;
   }
   public static void main(String[] args) {
      String s = "abcabcbb";
      System.out.println(lengthOfLongestSubstring(s)); // 3
   }
}
```

Logic :

1. Use sliding window with two pointers (start and i).
2. Maintain the last index of each character in an array.
3. If current character was seen inside the current window, move start to its next index.
4. Update the last index of current character.
5. Update max length as (i - start + 1).
6. Return the maximum length.

Output :

Input 1 :	abcabcbb	→	3 ("abc")
Input 2 :	bbbbb	→	1 ("b")
Input 3 :	pwwkew	→	3 ("wke")
Input 4 :	""	→	0

117 Valid Parentheses

Question : Given a string s containing parenthese '()', '{}', '[]', check if the input string is valid.
A string is valid if:
1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket.

Example :

Input : s = "()[]{}"
Output : true

Code (Java) :

```java
import java.util.*;
public class Main {
    public static boolean isValid(String s) {
        Stack <Character> st = new Stack<>();
        for (char ch : s.toCharArray()) {
            if (ch == '(' || ch == '{' || ch == '[') {
                st.push(ch);
            } else {
                if (st.isEmpty()) return false;
                char top = st.pop();
                if ((ch == ')' && top != '(') ||
                    (ch == '}' && top != '{') ||
                    (ch == ']' && top != '['))
                        return false;
            }
        }
        return st.isEmpty();
    }
    public static void main(String[] args) {
        String s = "()[]{}";
        System.out.println(isValid(s));  // true
    }
}
```

Logic :

1. Create an empty stack.
2. Traverse the string character by character.
3. If the character is an opening bracket ('(', '{', '['), push it onto the stack.
4. If the character is a closing bracket, pop from stack and check if it matches.
5. If at any time stack is empty when a closing bracket is found or mismatch occurs, return false.
6. After traversal, if stack is empty, return true (all brackets matched).

Output :

Input 1 :	s = "()[]{}"	→	true
Input 2 :	s = "(]"	→	false
Input 3 :	s = "(()]"	→	false
Input 4 :	s = "{{[]}}"	→	true

118 Roman to Integer

Question : Given a Roman numeral string s, convert it to an integer.

Example :

Input : s = "MCMXCIV"
Output : 1994

Code (Java) :

```java
import java.util.*;
public class Main {
    public static int romanToInt(String s) {
        Map<Character, Integer> map = new HashMap<>();
        map.put('I', 1);
        map.put('V', 5);
        map.put('X', 10);
        map.put('L', 50);
        map.put('C', 100);
        map.put('D', 500);
        map.put('M', 1000);
        int total = 0;
        for (int i = 0; i < s.length(); i++) {
            int curr = map.get(s.charAt(i));
            if (i + 1 < s.length() &&
                curr < map.get(s.charAt(i + 1))) {
                total -= curr;
            } else {
                total += curr;
            }
        }
        return total;
    }
    public static void main(String[] args) {
        String s = "MCMXCIV";
        System.out.println(romanToInt(s));  // 1994
    }
}
```

Logic :

1. Create a map of Roman characters and their values.
2. Traverse the string from left to right.
3. For each character, get its value.
4. If the current value is less than the next value, subtract it from total.
5. Otherwise, add it to total.
6. Return the final total.

Output :

Input 1 :	s = "MCMXCIV"	→	1994
Input 2 :	s = "LVIII"	→	58
Input 3 :	s = "III"	→	3
Input 4 :	s = "XLII"	→	42

119 Integer to Roman

Question : Convert an integer to its corresponding Roman numeral.

Example :

Input : 1994
Output : MCMXCIV

Code (Java) :

```java
public class Main {
  public static String intToRoman(int num) {
     int[] val = {1000, 900, 500, 400, 100, 90, 50,
          40, 10, 9, 5, 4, 1};
     String[] sym = {"M", "CM", "D", "CD", "C", "XC",
          "L", "XL", "X", "IX", "V", "IV", "I"};
     StringBuilder sb = new StringBuilder();
     for (int i = 0; i < val.length; i++) {
          while (num >= val[i]) {
               num -= val[i];
               sb.append(sym[i]);
          }
     }
      return sb.toString();
  }

  public static void main(String[] args) {
    int num = 1994;
    System.out.println(intToRoman(num)); // MCMXCIV
  }

}
```

Logic :

1. Create two arrays: one for integer values and another for corresponding Roman symbols.
2. Start from the largest value.
3. While the number is greater than or equal to the current value, append the symbol and subtract the value.
4. Move to the next value and repeat.
5. Continue until the number becomes 0.
6. Return the built Roman numeral.

Output :

Input 1 :	1994	→	MCMXCIV
Input 2 :	58	→	LVIII
Input 3 :	3	→	III
Input 4 :	42	→	XLII

120 Group Anagrams

Question : Given an array of strings, group the anagrams together.

Example :

Input : ["eat","tea","tan","ate","nat","bat"]
Output : [["eat","tea","ate"], ["tan","nat"], ["bat"]]

Code (Java) :

```java
import java.util.*;
public class Main {
  public static List<List<String>> groupAnagrams(String[] strs) {
     Map<String, List<String>> map = new HaskMap<>();
     for (String s : strs) {
        // sort the characters to form the key
        char[] arr = s.toCharArray();
        Arrays.sort(arr);
        String key = new String(arr);
        // add to the map
        map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
     }
     // return all grouped anagrams
     return new ArrayList<>(map.values());
  }

  public static void main(String[] args) {
     String[] strs = {"eat","tea","tan","ate","nat","bat"};
     List<List<String>> result = groupAnagrams(strs);
     System.out.println(result);
     // [[eat, tea, ate], [tan, nat], [bat]]
  }
}
```

Logic :

1. Anagrams contain the same characters with different orders.
2. Sort the characters of each string to create a key.
3. Use a HaskMap where key = sorted string and value = list of anagrams.
4. For each string, generate its key and add the string to the corresponding list.
5. After processing all strings, return all values from the map as the result.

Output :

```
Input 1 : ["eat","tea","tan","ate","nat","bat"]
       → [["eat","tea","ate"],["tan","nat"],["bat"]]
Input 2 : ["abc","bca","cab","xyz","zyx"]
       → [["abc","bca","cab"],["xyz","zyx"]]
Input 3 : ["a"]
       → [["a"]]
Input 4 : [""]
       → [[""]]
```

SECTION 5 – OOP & DSA Problems (121-150)

121 Student Class

Question : Create a Student class with attributes name, rollNo and marks. Include a method to display student details and another method to calculate average (out of 100).

Example :

Input : name = "Alice", rollNo = 101, marks = {85, 90, 78}
Output :
Name : Alice, Roll No : 101
Marks : [85, 90, 78]
Average : 84.33

Code (Java) :

```
class Student {
    String name;
    int rollNo;
    int[] marks;

    Student(String name, int rollNo, int[] marks) {
        this.name = name;
        this.rollNo = rollNo;
        this.marks = marks;
    }

    void display() {
        System.out.println("Name : " + name + ", Roll No : " + rollNo);
        System.out.print("Marks : [");
        for (int i = 0; i < marks.length; i++) {
            System.out.print(marks[i]);
            if (i < marks.length - 1) System.out.print(", ");
        }
        System.out.println("]");
        System.out.printf("Average : %.2f", average());
    }

    double average() {
        int sum = 0;
        for (int m : marks) sum += m;
        return (double) sum / marks.length;
    }
}
public class Main {
   public static void main(String[] args) {
      Student s = new Student("Alice", 101, new int[]{85, 90, 78});
      s.display();
   }
}
```

Logic :

1. Create a class Student with attributes name, rollNo, and marks array.
2. Initialize the values using a constructor.
3. display() prints student details and calls average() to calculate average.
4. average() computes the sum of marks and divides by number of subjects.

Output :

```
Name : Alice, Roll No : 101
Marks : [85, 90, 78]
Average : 84.33
```

122 Employee Class

Question : Create an Employee class with attributes empId, name, basicSalary. Include a method to calculate gross salary with 20% bonus and another method to display employee details.

Example :

Input : empId = 1, name = "John", basicSalary = 50000
Output :
ID : 1, Name : John, Basic Salary : 50000
Gross Salary (with 20% bonus) : 60000.0

Code (Java) :

```
class Employee {
    int empId;
    String name;
    double basicSalary;

    Employee(int empId, String name, double basicSalary) {
        this.empId = empId;
        this.name = name;
        this.basicSalary = basicSalary;
    }

    double grossSalary() {
        return basicSalary + (0.20 * basicSalary);
    }

    void display() {
        System.out.println("ID : " + empId + ", Name : " + name +
        ",  Basic Salary : " + basicSalary);
        System.out.println("Gross Salary (with 20% bonus) : " + grossSalary());
    }

}
public class Main {
   public static void main(String[] args) {
      Employee e = new Employee(1, "John", 50000);
      e.display();
   }
}
```

Logic :

1. Define class Employee with empId, name, and basicSalary.
2. Use constructor to initialize attributes.
3. grossSalary() adds 20% bonus to basic salary.
4. display() prints all details along with gross salary.

Output :

```
ID : 1, Name : John, Basic Salary : 50000
Gross Salary (with 20% bonus) : 60000.0
```

123. Bank Account Class

Question: Create a BankAccount class with accountNumber, accountHolder, and balance as attributes. Implement methods to deposit, withdraw and display account details.

Code:

```
class BankAccount {
    private String accountNumber;
    private String accountHolder;
    private double balance;

    public BankAccount(String accNo, String holder, double bal) {
        this.accountNumber = accNo;
        this.accountHolder = holder;
        this.balance = bal;
    }

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Deposited: " + amount);
        } else {
            System.out.println("Invalid deposit amount!");
        }
    }

    public void withdraw(double amount) {
        if (amount > 0) && amount <= balance) {
            balance -= amount;
            System.out.println("Withdrawn: " + amount);
        } else {
            System.out.println("Invalid or insufficient balance!");
        }
    }

    public void display() {
        System.out.println("Account Number : " + accountNumber);
        System.out.println("Account Holder : " + accountHolder);
        System.out.println("Balance  :  " + balance);
    }
}
public class Main {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount("BA123", "Rahul", 5000.0);
        acc.display();
        acc.deposit(1500.0);
        acc.withdraw(1200.0);
        acc.display();
    }
}
```

Logic Explanation:

1. Create a class BankAccount with private attributes.
2. Use constructor to initialize account details.
3. deposit() method adds amount to balance if amount > 0.
4. withdraw() method subtracts amount if sufficient balance is available.
5. display() method prints the account details.

Output:

```
Account Number : BA123
Account Holder : Rahul
Balance        : 5000.0
Deposited: 1500.0
Withdrawn: 1200.0
Account Number : BA123
Account Holder : Rahul
Balance : 5300.0
```

124. Constructor Overloading

Question: Create a class Student with id, name and marks. Use constructor overloading to initialize objects with different parameters.

Code:

```
class Student {
    int id;
    String name;
    double marks;

    // Constructor 1: No-arg constructor
    Student() {
        id = 0;
        name = "Unknown";
        marks = 0.0;
    }
    // Constructor 2: Parameterized constructor (id, name)
    Student(int id, String name) {
        this.id = id;
        this.name = name;
        this.marks = 0.0;
    }
    // Constructor 3: Parameterized constructor (id, name, marks)
    Student(int id, String name, double marks) {
        this.id = id;
        this.name = name;
        this.marks = marks;
    }
    void display() {
        System.out.println("ID: " + id + ", Name: " + name
                           + ", Marks: " + marks);
    }
}
public class Main {
   public static void main(String[] args) {
       Student s1 = new Student();
       Student s2 = new Student(101, "Alice");
       Student s3 = new Student(102, "Bob", 88.5);

       s1.display();
       s2.display();
       s3.display();
   }
}
```

Logic Explanation:

1. Constructor overloading means having multiple constructors with different parameter lists.
2. No-arg constructor initializes default values.
3. Second constructor initializes id and name, marks is set to 0.0.
4. Third constructor initializes all the fields.
5. Appropriate constructor is called based on the arguments passed while creating an object.

Output:

```
ID: 0, Name: Unknown, Marks: 0.0
ID: 101, Name: Alice, Marks: 0.0
ID: 102, Name: Bob, Marks: 88.5
```

125. Method Overloading

Question: Create a class Calculator that has an overloaded method add(). One method should add two integers, another should add three integers, and another should add two double values.

Code:

```
class Calculator {
    public int add(int a, int b) {
        return a + b;
    }

    public int add(int a, int b, int c) {
        return a + b + c;
    }

    public double add(double a, double b) {
        return a + b;
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator calc = new Calculator();

        System.out.println("Add(10, 20) = " + calc.add(10, 20));
        System.out.println("Add(10, 20, 30) = " + calc.add(10, 20, 30));
        System.out.println("Add(12.5, 7.5) = " + calc.add(12.5, 7.5));
    }
}
```

Logic Explanation:

1. Method overloading means having multiple methods with the same name in the same class.
2. The methods must have different parameter lists (different type, number or order of parameters).
3. The correct method is chosen at compile-time based on the arguments passed.
4. Here, three add() methods are defined with different parameters, so the appropriate method is called for each call.

Output:

```
Add(10, 20) = 30
Add(10, 20, 30) = 60
Add(12.5, 7.5) = 20.0
```

126. Method Overriding

Question: Create a base class Animal with a method sound(). Create a derived class Dog that overrides sound() method to provide a specific implementation.

Code:

```
class Animal {
    public void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    public void sound() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a = new Animal();
        Animal d = new Dog();

        a.sound();      // calls Animal's sound()
        d.sound();      // calls Dog's overridden sound()
    }
}
```

Logic Explanation:

1. Method overriding occurs when a subclass provides a specific implementation of a method already defined in its superclass.
2. The method in the subclass must have the same name, return type, and parameters as in the superclass.
3. The @Override annotation is used to ensure we are actually overriding a method.
4. The method to be executed is decided at runtime based on the object's actual type (runtime polymorphism).

Output:

```
Animal makes a sound
Dog barks
```

* **Key Difference:**

  Overloading → Same class, same method name, different parameters, compile-time.

  Overriding → Parent & child class, same method name & signature, runtime.

127. Single Inheritance

Question: Create a class Person with name and age. Create a class Student that inherits from Person and adds rollNo. Display all the details.

Code:

```
// Parent class
class Person {
    String name;
    int age;

    void displayPerson() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}

// Child class
class Student extends Person {
    int rollNo;

    void displayStudent() {
        displayPerson(); // calling parent method
        System.out.println("Roll No: " + rollNo);
    }
}

// Main class
public class Main {
    public static void main(String[] args) {
        Student s = new Student();
        s.name = "Alice";
        s.age = 20;
        s.rollNo = 101;

        s.displayStudent();
    }
}
```

Logic Explanation:

1. Person is the parent (superclass) with fields name and age.
2. Student is the child (subclass) and inherits all fields and methods of Person.
3. Student adds its own field rollNo and a method displayStudent().
4. Using an object of Student, we can access parent members and child members.
5. This is called Single Inheritance because one child class inherits from one parent class.

Output:

```
Name: Alice, Age: 20
Roll No: 101
```

128. Multilevel Inheritance

Question: Create three classes:

1. Grandparent (with a method showGrand())
2. Parent (inherits Grandparent, with showParent())
3. Child (inherits Parent, with showChild())

Create object of Child and call all methods.

Code:

```
class Grandparent {
    void showGrand() {
        System.out.println("This is Grandparent class");
    }
}
class Parent extends Grandparent {
    void showParent() {
        System.out.println("This is Parent class");
    }
}
class Child extends Parent {
    void showChild() {
        System.out.println("This is Child class");
    }
}

public class Main {
    public static void main(String[] args) {
        Child c = new Child();
        c.showGrand();    // inherited from Grandparent
        c.showParent();   // inherited from Parent
        c.showChild();    // own method of Child
    }
}
```

Logic Explanation:

1. Grandparent is the top-most class.
2. Parent inherits Grandparent.
3. Child inherits Parent (and indirectly Grandparent).
4. Child object can access methods from its class, parent class, and grandparent class.
5. This is called Multilevel Inheritance because inheritance occurs in a chain (Grandparent → Parent → Child).

Output:

```
This is Grandparent class
This is Parent class
This is Child class
```

* **Key Difference:**

  Single Inheritance → One child class inherits from one parent class.

  Multilevel Inheritance → A class inherits from a class, which is already inherited from another class.

129. Encapsulation Example

Question: Create a class Account that uses encapsulation. The class should have private data members and public getter and setter methods to access and modify the data.

Code:

```
class Account {
    // private data members (encapsulated)
    private int accountNumber;
    private String accountHolder;
    private double balance;

 // constructor
 public Account(int accountNumber, String accountHolder, double balance) {
     this.accountNumber = accountNumber;
     this.accountHolder = accountHolder;
     this.balance = balance;
 }

 // getter methods (read access)
 public int getAccountNumber() {
     return accountNumber;
 }

 public String getAccountHolder() {
     return accountHolder;
 }

 public double getBalance() {
     return balance;
 }

 // setter methods (write access with validation)
 public void setAccountHolder(String accountHolder) {
     this.accountHolder = accountHolder;
 }
 public void setBalance(double balance) {
     if (balance >= 0) {
         this.balance = balance;
     } else {
         System.out.println("Balance cannot be negative!");
     }
 }
 // business method
 public void deposit(double amount) {
     if (amount > 0) {
         balance += amount;
     } else {
         System.out.println("Deposit amount must be positive!");
     }
 }

 public void withdraw(double amount) {
     if (amount > 0 && amount <= balance) {
         balance -= amount;
     } else {
         System.out.println("Insufficient balance or invalid amount!");
     }
 }

 public void displayAccount() {
     System.out.println("Account Number : " + accountNumber);
     System.out.println("Account Holder : " + accountHolder);
     System.out.println("Balance : " + balance);
 }
}
public class Main {
 public static void main(String[] args) {
     Account acc = new Account(1001, "Rahul Sharma", 5000.0);
     System.out.println("--- Initial Account Details ---");
     acc.displayAccount();

     System.out.println("\n--- After Deposit 2000 ---");
     acc.deposit(2000);
     acc.displayAccount();

     System.out.println("\n--- After Withdraw 1500 ---");
     acc.withdraw(1500);
     acc.displayAccount();

     System.out.println("\n--- Try Invalid Withdraw (6000) ---");
     acc.withdraw(6000);
     acc.displayAccount();
 }
}
```

Logic Explanation:

1. All data members (accountNumber, accountHolder, balance) are declared private, so they cannot be accessed directly from outside the class.
2. Public getter methods provide read access to the data.
3. Public setter methods provide controlled write access with validation (e.g., balance cannot be negative).
4. deposit() and withdraw() are business methods that modify the balance safely.
5. This is encapsulation — hiding the internal state and exposing only what is necessary through public methods.

Output:

```
--- Initial Account Details ---
Account Number : 1001
Account Holder : Rahul Sharma
Balance : 5000.0

--- After Deposit 2000 ---
Account Number : 1001
Account Holder : Rahul Sharma
Balance : 7000.0

--- After Withdraw 1500 ---
Account Number : 1001
Account Holder : Rahul Sharma
Balance : 5500.0

--- Try Invalid Withdraw (6000) ---
Insufficient balance or invalid amount!
Account Number : 1001
Account Holder : Rahul Sharma
Balance : 5500.0
```

*** Key Point:**

Encapsulation improves data security by restricting direct access to the data and allows control over how the data is modified.

130. Abstraction Example

Question: Create an abstract class Shape with an abstract method area(). Create two subclasses Circle and Rectangle that implement the area() method and display the area.

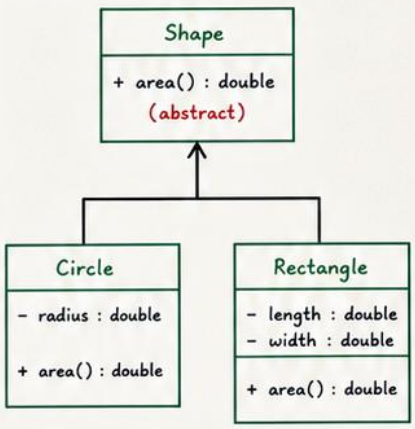
Code:

```
// Abstract class
abstract class Shape {
    // Abstract method (no body)
    public abstract double area();
}

// Circle class
class Circle extends Shape {
    private double radius;

    public Circle(double r) {
        this.radius = r;
    }

    // Implementing abstract method
    @Override
    public double area() {
        return Math.PI * radius * radius;
    }
}

// Rectangle class
class Rectangle extends Shape {
    private double length, width;

    public Rectangle(double l, double w) {
        this.length = l;
        this.width = w;
    }

    // Implementing abstract method
    @Override
    public double area() {
        return length * width;
    }
}

// Main class to test
public class Main {
    public static void main(String[] args) {
        Shape s1 = new Circle(5);
        Shape s2 = new Rectangle(4, 6);

        System.out.println("Area of Circle    : " + s1.area());
        System.out.println("Area of Rectangle : " + s2.area());
    }
}
```

Logic Explanation:

1. We create an abstract class Shape that contains an abstract method area().
2. An abstract method has no body in the abstract class.
3. Circle and Rectangle are subclasses that extend Shape.
4. Both subclasses provide their own implementation of area().
5. We cannot create an object of Shape (abstract class).
6. We create objects of Circle and Rectangle and call area() using the reference of Shape.

UML Diagram:



Output:

```
Area of Circle    : 78.53981633974483
Area of Rectangle : 24.0
```

Key Points:

- Abstraction means showing only essential features and hiding implementation details.
- Achieved in Java using abstract class and interface.
- Abstract class can have abstract methods (without body) and concrete methods (with body).
- Subclasses must provide implementation for all abstract methods.

Note: Abstraction helps in reducing complexity and makes the code more maintainable and secure.

131. Interface Example

Question: Create an interface Payable with a method calculatePay(). Create two classes Employee and Invoice that implement this interface and provide their own implementation.

Code:

```java
// Interface
interface Payable {
    double calculatePay();
}

// Employee class
class Employee implements Payable {
    private String name;
    private double salary;

    public Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    @Override
    public double calculatePay() {
        return salary;   // Employee pay is salary
    }
    public String getName() { return name; }
}

// Invoice class
class Invoice implements Payable {
    private String item;
    private int quantity;
    private double price;

    public Invoice(String item, int quantity, double price) {
        this.item = item;
        this.quantity = quantity;
        this.price = price;
    }
    @Override
    public double calculatePay() {
        return quantity * price;   // Total amount of invoice
    }
    public String getItem() { return item; }
}

// Main class to test
public class Main {
    public static void main(String[] args) {
        Payable p1 = new Employee("Alice", 50000);
        Payable p2 = new Invoice("Laptop", 2, 25000.0);

        System.out.println("Employee Name : " + ((Employee)p1).getName());
        System.out.println("Employee Pay  : " + p1.calculatePay());

        System.out.println("Item Name     : " + ((Invoice)p2).getItem());
        System.out.println("Invoice Amount : " + p2.calculatePay());
    }
}
```

Logic Explanation:

1. We define an interface Payable with a method calculatePay().
2. Employee class implements Payable and returns the employee's salary as pay.
3. Invoice class implements Payable and returns total amount (quantity × price) as pay.
4. In the Main class, we create objects of both classes and access them using Payable reference (interface).
5. This shows how interface enables multiple classes to provide their own implementation of the same method.

Output:

```
Employee Name : Alice
Employee Pay  : 50000.0
Item Name     : Laptop
Invoice Amount : 50000.0
```

Key Points:

- An interface in Java contains abstract methods (by default public and abstract).
- A class implements an interface using the 'implements' keyword.
- A class must provide the implementation for all methods of the interface.
- Interface allows 100% abstraction (only method declaration, no body).
- Achieves loose coupling and supports multiple implementations.

Note: Unlike inheritance, a class can implement multiple interfaces.

132. Runtime Polymorphism

Question: Create a base class Animal with a method sound(). Create two derived classes Dog and Cat that override the sound() method. Demonstrate runtime polymorphism using a base class reference.

Code:

```
// Base class
class Animal {
    public void sound() {
        System.out.println("Animal makes a sound");
    }
}

// Derived class Dog
class Dog extends Animal {
    @Override
    public void sound() {
        System.out.println("Dog barks");
    }
}

// Derived class Cat
class Cat extends Animal {
    @Override
    public void sound() {
        System.out.println("Cat meows");
    }
}

// Main class to test runtime polymorphism
public class Main {
    public static void main(String[] args) {
        Animal a;       // Base class reference
        a = new Dog();  // Dog object
        a.sound();      // Calls Dog's sound()

        a = new Cat();  // Cat object
        a.sound();      // Calls Cat's sound()
    }
}
```

Logic Explanation:

1. The base class Animal has a method sound().
2. Dog and Cat classes override the sound() method.
3. In the Main class, we create a reference variable of type Animal.
4. At runtime, the JVM determines the actual object type and calls the overridden method.
5. This is called Runtime Polymorphism (Method Overriding).

How it works?

The method call is resolved at runtime based on the actual object referred by the base class reference.

Output:

```
Dog barks
Cat meows
```

Step-by-step Execution:

1. Animal a;
2. a = new Dog(); // a refers to Dog object
3. a.sound(); // JVM calls Dog's sound() → Dog barks
4. a = new Cat(); // a now refers to Cat object
5. a.sound(); // JVM calls Cat's sound() → Cat meows

Key Points:

- Runtime polymorphism is achieved through method overriding.
- The method to be executed is decided at runtime (not compile-time).
- It allows one interface (method) to have multiple implementations.
- It provides flexibility and extensibility in programs.

Note: Runtime Polymorphism = Method Overriding + Dynamic Method Dispatch (Decision taken by JVM at runtime)

133. Singleton Class

Question: Create a Singleton class Database that ensures only one object is created. Provide a method getConnection() to simulate getting a database connection. Demonstrate that only one instance is created.

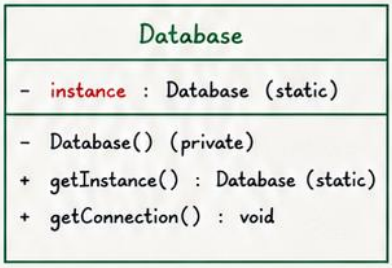
Code:

```
// Singleton class
class Database {
    // private static instance (only one object)
    private static Database instance;

    // private constructor (prevents direct instantiation)
    private Database() {
        System.out.println("Database object created");
    }

    // public method to get the single instance
    public static Database getInstance() {
        if (instance == null) {
            instance = new Database();  // create only once
        }
        return instance;
    }

    // example method
    public void getConnection() {
        System.out.println("Getting database connection...");
    }
}

// Main class to test Singleton
public class Main {
    public static void main(String[] args) {
        Database db1 = Database.getInstance();
        db1.getConnection();

        Database db2 = Database.getInstance();
        db2.getConnection();

        System.out.println("db1 == db2 ? " + (db1 == db2));
    }
}
```

Logic Explanation:

1. The constructor of Database is private, so no other class can create an object directly.
2. A private static variable instance holds the only object of the class.
3. The public static method getInstance() checks if instance is null.
4. If null, a new object is created and stored in instance.
5. If not null, the existing object is returned.
6. Thus, only one object is ever created (Singleton).

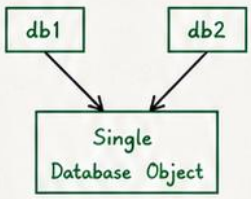
Singleton Class Diagram:



Only one instance of Database is created and shared globally.

Output:

```
Database object created
Getting database connection...
Getting database connection...
db1 == db2 ? true
```



Both db1 and db2 reference the same object in memory.

Key Points:

- Singleton pattern ensures a class has only one instance.
- Used when exactly one object is needed to coordinate actions across the system.
- Common use cases: Database connections, Configuration settings, Logging, Caching.
- It saves memory and provides a global point of access.

Note: This is a basic Singleton implementation. In multi-threaded environments, additional synchronization may be needed to make it thread-safe.

134. Immutable Class

Question: Create an immutable class Person with fields name and age. Provide constructor and getter methods. Demonstrate that once an object is created, its state cannot be changed.

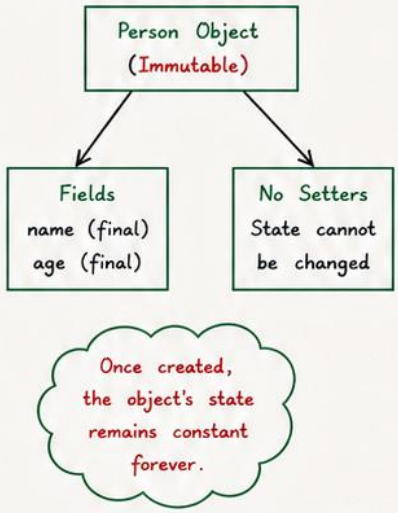
Code:

```
// Immutable class Person
final class Person {
    // final fields
    private final String name;
    private final int age;

    // Constructor
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Getter methods
    public String getName() {
        return name;
    }
    public int getAge() {
        return age;
    }

    // No setter methods → ensures immutability
}

// Main class to test
public class Main {
   public static void main(String[] args) {
       Person p1 = new Person("Alice", 25);

       System.out.println("Name : " + p1.getName());
       System.out.println("Age  : " + p1.getAge());

       // Attempting to change values (Not allowed)
       // p1.name = "Bob";  // Compile-time Error
       // p1.age = 30;      // Compile-time Error

       // Creating another object (new state)
       Person p2 = new Person("Alice", 26);
       System.out.println("New Age : " + p2.getAge());
   }
}
```

Logic Explanation:

1. The class Person is declared final so it cannot be inherited.
2. The fields name and age are declared private and final, so their values cannot be changed once initialized.
3. Only a constructor is provided to set values at the time of object creation.
4. Getter methods provide read-only access to the fields.
5. No setter methods are provided, so the state of the object cannot be modified after creation.
6. If we need a different state, we must create a new object.

Immutability Diagram:



Output:

```
Name    : Alice
Age     : 25
New Age : 26
```

Explanation of Output:

- The object p1 is created with name = "Alice" and age = 25.
- Its values are printed using getter methods.
- We cannot change the values of p1 (compile-time error if attempted).
- A new object p2 is created with a different state (age = 26).

Key Points:

- Immutable class means the object's state cannot be modified after creation.
- Achieved using final class, final fields, and no setter methods.
- Enhances security, thread-safety, and reliability.
- Commonly used for value objects like String, wrapper classes (Integer, Double), LocalDate, etc.

Note: Immutability is a good practice in Java for creating secure and predictable classes.

135. Object Cloning

Question: Create a class Employee with fields id and name. Implement a method clone() that returns a copy of the current object. Demonstrate cloning.

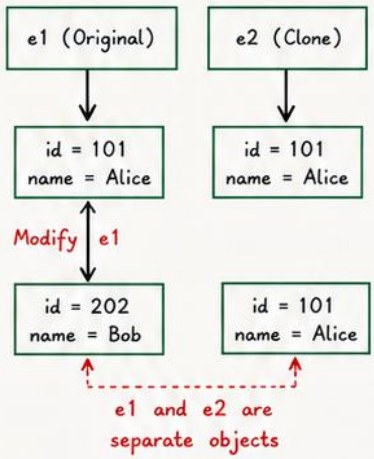
Code:

```
// Employee class that supports cloning
class Employee implements Cloneable {
    private int id;
    private String name;

    // Constructor
    public Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    // Getter methods
    public int getId() { return id; }
    public String getName() { return name; }

    // Overriding clone() method
    @Override
    public Employee clone() {
        try {
            return (Employee) super.clone(); // field-by-field copy
        } catch (CloneNotSupportedException e) {
            throw new AssertionError();
        }
    }
    @Override
    public String toString() {
        return "Employee [id=" + id + ", name=" + name + "]";
    }
}
// Main class to test cloning
public class Main {
    public static void main(String[] args) {
        Employee e1 = new Employee(101, "Alice");
        Employee e2 = e1.clone(); // clone of e1

        System.out.println("Original: " + e1);
        System.out.println("Cloned  : " + e2);

        // Change original object
        e1.name = "Bob";
        e1.id = 202;

        System.out.println("\nAfter modifying original:");
        System.out.println("Original: " + e1);
        System.out.println("Cloned  : " + e2);
    }
}
```

Logic Explanation:

1. The class Employee implements the Cloneable interface.
2. The clone() method is overridden to return a copy of the current object using super.clone().
3. The clone() performs a shallow copy (field-by-field copy).
4. In the main method, e2 is a clone of e1.
5. After modifying e1, e2 remains unchanged, proving a separate copy was created.

Diagram (Cloning):



Output:

```
Original: Employee [id=101, name=Alice]
Cloned  : Employee [id=101, name=Alice]

After modifying original:
Original: Employee [id=202, name=Bob]
Cloned  : Employee [id=101, name=Alice]
```

Key Points:

- Object cloning creates a copy of an object.
- The cloned object is independent of the original.
- Default clone() creates a shallow copy.
- For deep copy, mutable fields (like lists, arrays) must be cloned explicitly.
- Cloneable is a marker interface (no methods).

Note: Cloneable is just a marker interface. It does not contain any methods. Overriding clone() and calling super.clone() is the standard way to clone an object in Java.

136. Stack Using Array

Question: Implement a Stack using array. Support operations push(), pop(), peek() and isEmpty(). Also handle overflow and underflow conditions.

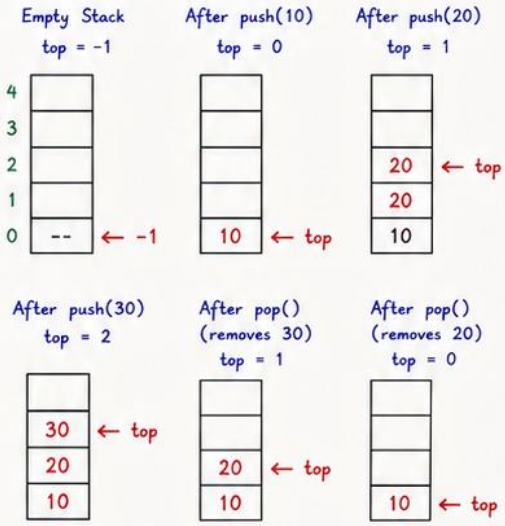
Code:

```java
// Stack implementation using Array
class StackArray {
    private int[] arr;    // array to store stack elements
    private int top;      // index of top element
    private int capacity; // maximum size of stack

    // Constructor
    public StackArray(int size) {
        capacity = size;
        arr = new int[capacity];
        top = -1;     // stack is initially empty
    }

    // Push operation
    public void push(int x) {
        if (top == capacity - 1) {
            System.out.println("Stack Overflow!");
            return;
        }
        arr[++top] = x;   // increment top and insert
        System.out.println(x + " pushed");
    }

    // Pop operation
    public int pop() {
        if (top == -1) {
            System.out.println("Stack Underflow!");
            return -1;
        }
        int x = arr[top--];   // return top and decrement
        return x;
    }

    // Peek operation
    public int peek() {
        if (top == -1) {
            System.out.println("Stack Underflow!");
            return -1;
        }
        return arr[top];    // return top element
    }

    // isEmpty operation
    public boolean isEmpty() {
        return top == -1;
    }
}
// Main class to test the stack
public class Main {
    public static void main(String[] args) {
        StackArray st = new StackArray(5);
        st.push(10);
        st.push(20);
        st.push(30);
        System.out.println("Top element: " + st.peek());
        System.out.println("Popped: " + st.pop());
        System.out.println("Top element: " + st.peek());
        System.out.println("Is stack empty? " + st.isEmpty());
        st.pop();
        st.pop();
        st.pop();    // one extra pop to show underflow
        System.out.println("Is stack empty? " + st.isEmpty());
    }
}
```

Logic Explanation:

1. We use an integer array 'arr' to store stack elements.
2. A variable 'top' keeps track of the top element index. Initially top = -1 (empty).
3. Push: Check overflow (top == capacity-1). If not, increment top and insert element.
4. Pop: Check underflow (top == -1). If not, return element at top and decrement top.
5. Peek: Return element at top without removing it.
6. isEmpty: If top == -1, stack is empty.

Diagram (Stack State)



Output:

```
10 pushed
20 pushed
30 pushed
Top element: 30
Popped: 30
Top element: 20
Is stack empty? false
Popped: 20
Popped: 10
Stack Underflow!
Is stack empty? true
```

Key Points:

- Stack follows LIFO (Last In First Out) principle.
- Push and Pop operations take O(1) time.
- Overflow occurs when stack is full (top == capacity-1).
- Underflow occurs when stack is empty (top == -1).
- Array based stack has fixed size.

Note: This is a linear data structure. For dynamic size, use Stack using Linked List.

137. Queue Using Array

Question: Implement a Queue using array. Support operations enqueue(), dequeue(), peek() and isEmpty(). Also handle overflow condition.

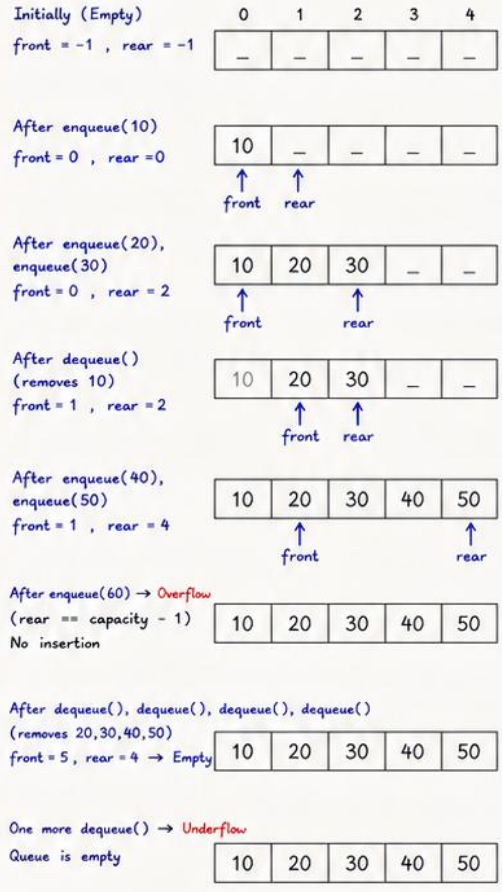
Code:

```
// Queue implementation using Array
class QueueArray {
    private int[] arr;      // array to store queue elements
    private int front;      // index of front element
    private int rear;       // index of rear element
    private int capacity;   // maximum size

    // Constructor
    public QueueArray(int size) {
        capacity = size;
        arr = new int[capacity];
        front = -1;    // queue is initially empty
        rear = -1;
    }

    // Enqueue operation
    public void enqueue(int x) {
        if (rear == capacity - 1) {
            System.out.println("Queue Overflow!");
            return;
        }
        if (front == -1) front = 0;    // first element
        arr[++rear] = x;    // insert and increment rear
        System.out.println(x + " enqueued");
    }
    // Dequeue operation
    public int dequeue() {
        if (isEmpty()) {
            System.out.println("Queue Underflow!");
            return -1;
        }
        int x = arr[front++];
        if (front > rear) {    // queue became empty
            front = rear = -1;
        }
        System.out.println(x + " dequeued");
        return x;
    }
    // Peek operation
    public int peek() {
        if (isEmpty()) {
            System.out.println("Queue is empty");
            return -1;
        }
        return arr[front];    // front element
    }
    // isEmpty operation
    public boolean isEmpty() {
        return front == -1;
    }
}
// Main class to test queue
public class Main {
    public static void main(String[] args) {
        QueueArray q = new QueueArray(5);
        q.enqueue(10);
        q.enqueue(20);
        q.enqueue(30);
        System.out.println("Front element: " + q.peek());
        q.dequeue();
        q.enqueue(40);
        q.enqueue(50);
        q.dequeue(60);      // this will cause overflow
        q.dequeue();
        q.dequeue();
        q.dequeue();
        q.dequeue();        // this will cause underflow
    }
}
```

Logic Explanation:

1. We use an array to store queue elements.
2. 'front' points to the first element. 'rear' points to the last element.
3. Initially front = rear = -1 (empty queue).
4. Enqueue:
   - If rear == capacity - 1 → Overflow.
   - If first element, set front = 0.
   - Insert element at ++rear.
5. Dequeue:
   - If queue is empty → Underflow.
   - Remove element at front and do front++.
   - If front > rear → queue becomes empty, set front = rear = -1.
6. Peek: Return element at front without removing.
7. isEmpty: Check if front == -1.

Diagram (Queue State):



Output:

```
10 enqueued
20 enqueued
30 enqueued
Front element: 10
10 dequeued
40 enqueued
50 enqueued
Queue Overflow!
20 dequeued
30 dequeued
40 dequeued
50 dequeued
Queue Underflow!
```

Key Points:

- Queue follows FIFO (First In First Out) principle.
- Enqueue and Dequeue operations take O(1) time.
- Overflow occurs when rear == capacity - 1.
- Underflow occurs when queue is empty (front == -1).
- Array based queue has fixed size.

Note: For dynamic size, use Queue using Linked List.

138. Linked List Insertion

Question: Implement a Singly Linked List and write functions to insert a node at the beginning, at the end and at a specific position.

Code:

```
// Node class
class Node {
      int data;
      Node next;
      Node(int data) {
         this.data = data;
         this.next = null;
      }
}
// Linked List class
class LinkedList {
   Node head;

   // Insert at beginning
   public void insertAtBeginning(int data) {
       Node newNode = new Node(data);
       newNode.next = head;
       head = newNode;
   }
   // Insert at end
   public void insertAtEnd(int data) {
        Node newNode = new Node(data);
        if (head == null) {
            head = newNode;
            return;
        }
        Node temp = head;
        while (temp.next != null) {
            temp = temp.next;
        }
        temp.next = newNode;
   }

   // Insert at position (1-based index)
   public void insertAtPosition(int data, int pos) {
        if (pos <= 1) {
            insertAtBeginning(data);
            return;
        }
        Node newNode = new Node(data);
        Node temp = head;
        for (int i = 1; temp != null && i < pos - 1; i++) {
            temp = temp.next;
        }
        if (temp == null) { // position > length+1
            System.out.println("Invalid Position");
            return;
        }
        newNode.next = temp.next;
        temp.next = newNode;
   }
   // Display list
   public void display() {
        Node temp = head;
        while (temp != null) {
            System.out.print(temp.data + " -> ");
            temp = temp.next;
        }
        System.out.println("NULL");
   }
}
// Main class to test
public class Main {
   public static void main(String[] args) {
      LinkedList list = new LinkedList();
      list.insertAtEnd(10);
      list.insertAtEnd(20);
      list.insertAtEnd(30);
      System.out.print("Initial List    : ");
      list. display();
      list.insertAtBeginning(5);
      System.out.print("After insert at beginning (5): ");
      list. display();
      list.insertAtEnd(40);
      System.out.print("After insert at end (40)    : ");
      list. display();
      list.insertAtPosition(25, 3);
      System.out.print("After insert 25 at pos 3   : ");
      list. display();
   }
}
```

Logic Explanation:

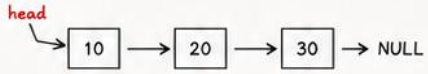
① **Insert at Beginning:**
- Create new node.
- Point new node's next to current head.
- Move head to new node.

② **Insert at End:**
- Create new node.
- If list is empty, make new node as head.
- Else, traverse to last node and link the new node at the end.

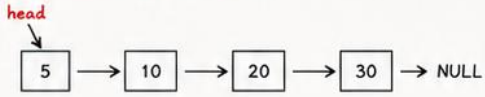
③ **Insert at Position (1-based index):**
- If position <= 1, insert at beginning.
- Else, traverse to (pos-1)th node.
- Insert new node after it.
- If position > length+1 → Invalid.

Visual Diagrams:

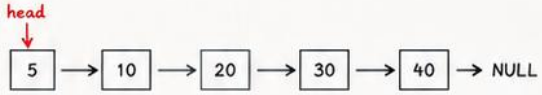
Initial List:



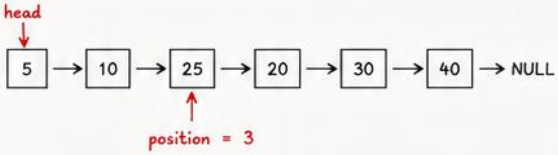
After insert at beginning (5):



After insert at end (40):



After insert 25 at position 3:



Output:

```
Initial List                : 10 -> 20 -> 30 -> NULL
After insert at beginning (5): 5 -> 10 -> 20 -> 30 -> NULL
After insert at end (40)    : 5 -> 10 -> 20 -> 30 -> 40 -> NULL
After insert 25 at pos 3    : 5 -> 10 -> 25 -> 20 -> 30 -> 40 -> NULL
```

Key Points:

- Insertion at beginning takes O(1) time.
- Insertion at end requires O(n) time (traverse to last).
- Insertion at position requires O(n) time.
- Always handle empty list and invalid positions.

Note: This is a Singly Linked List. For Doubly Linked List, each node contains a previous pointer also.

140. Binary Tree Traversal

Question: Given a binary tree, perform the following traversals:

1. Preorder 2. Inorder 3. Postorder 4. Level Order (BFS)

Code:

```
// Node class
class Node {
      int data;
      Node left, right;
      Node(int data) {
         this.data = data;
         left = right = null;
      }
}

// Binary Tree Traversal
class BinaryTree {
      Node root;

   // Preorder: Root → Left → Right
   void preorder(Node node) {
      if (node == null) return;
      System.out.print(node.data + " ");
      preorder(node.left);
      preorder(node.right);
   }

   // Inorder: Left → Root → Right
   void inorder(Node node) {
      if (node == null) return;
      inorder(node.left);
      System.out.print(node.data + " ");
      inorder(node.right);
   }
   // Postorder: Left → Right → Root
   void postorder(Node node) {
      if (node == null) return;
      postorder(node.left);
      postorder(node.right);
      System.out.print(node.data + " ");
   }
   // Level Order (BFS)
   void levelOrder() {
      if (root == null) return;
      Queue<Node> q = new LinkedList<();
      q.add(root);
      while (!q.isEmpty()) {
         Node curr = q.poll();
         System.out.print(curr.data + " ");
         if (curr.left != null) q.add(curr.left);
         if (curr.right != null) q.add(curr.right);
      }
   }
   public static void main(String[] args) {
      BinaryTree tree = new BinaryTree();
      /* Construct the following tree
                  1
                /   \
               2     3
              / \   / \
             4   5 6   7 */
      tree.root = new Node(1);
      tree.root.left = new Node(2);
      tree.root.right = new Node(3);
      tree.root.left.left = new Node(4);
      tree.root.left.right = new Node(5);
      tree.root.right.left = new Node(6);
      tree.root.right.right = new Node(7);
      System.out.print("Preorder: ");
      tree.preorder(tree.root);
      System.out.println();
      System.out.print("Inorder: ");
      tree.inorder(tree.root);
      System.out.println();
      System.out.print("Postorder: ");
      tree.postorder(tree.root);
      System.out.println();
      System.out.print("Level Order: ");
      tree.levelOrder();
   }
}
```

Logic Explanation:

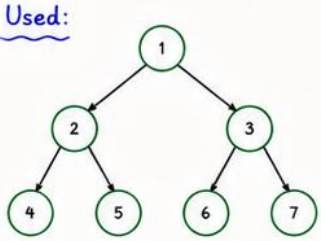
① **Preorder (Root → Left → Right)**
- Visit the root.
- Traverse the left subtree.
- Traverse the right subtree.

② **Inorder (Left → Root → Right)**
- Traverse the left subtree.
- Visit the root.
- Traverse the right subtree.

③ **Postorder (Left → Right → Root)**
- Traverse the left subtree.
- Traverse the right subtree.
- Visit the root.

④ **Level Order (Breadth First Search)**
- Use a queue.
- Visit nodes level by level from left to right.
- Enqueue left child first, then right child.

Tree Used:



Dry Run (Level Order):

Initial: Queue = [1]

Visit 1:	1	Output: 1
Visit 2:	2 3	Output: 1 2
Visit 2:	3 4 5	Output: 1 2 3
Visit 4:	4 5 6 7	Output: 1 2 3 4
Visit 5:	5 6 7	Output: 1 2 3 4 5
Visit 6:	6 7	Output: 1 2 3 4 5 6
Visit 7:	7	Output: 1 2 3 4 5 6 7
	(empty)	

Output:

```
Preorder:     1 2 4 5 3 6 7
Inorder:      4 2 5 1 6 3 7
Postorder:    4 5 2 6 7 3 1
Level Order:  1 2 3 4 5 6 7
```

Key Points:

- Preorder is useful for copying a tree.
- Inorder gives sorted order for BST.
- Postorder is useful for deleting a tree.
- Level order uses BFS with a queue.
- Time Complexity for all: O(n)
- Space Complexity: O(h) for recursion (h = height of tree) and O(w) for level order (w = max width)

Note: Tree Traversal is a fundamental concept in DSA and is widely used in coding interviews.

141. BUBBLE SORT

* Question:

Write a program to sort an array using Bubble Sort algorithm in ascending order.

* Code (Java):

```java
import java.util.*;
public class BubbleSort {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter size of array: ");
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.println("Enter elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        bubbleSort(arr, n);
        System.out.println("Sorted array:");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
    }
    static void bubbleSort(int[] arr, int n) {
        for (int i = 0; i < n-1; i++) {
            boolean swapped = false;
            for (int j = 0; j < n-i-1; j++) {
                if (arr[j] > arr[j+1]) {
                    // swap
                    int temp = arr[j];
                    arr[j] = arr[j+1];
                    arr[j+1] = temp;
                    swapped = true;
                }
            }
            // If no two elements were swapped in inner loop,
               the array is already sorted.
            if (!swapped) break;
        }
    }
}
```

* Code Logic Explanation:

1. Start from the first element and compare it with the next element.
2. If the current element is greater than the next element, swap them.
3. After one pass, the largest element becomes the last element.
4. Repeat the process for the remaining unsorted elements.
5. Use a boolean variable 'swapped' to stop early if no swaps occur in a pass (array is already sorted).

* Output:

```
Enter size of array: 5
Enter elements:
64 34 25 12 22
Sorted array:
12 22 25 34 64
```

Passes (for above example):

```
Pass 1: 34 25 12 22 64
Pass 2: 25 12 22 34 64
Pass 3: 12 22 25 34 64
Pass 4: 12 22 25 34 64 (no swap)
        ⇒ Sorted
```

* Time Complexity: $O(n^2)$ | Space Complexity: $O(1)$

142. SELECTION SORT

* **Question:**

Write a program to sort an array using Selection Sort algorithm in ascending order.

* **Code (Java):**

```
import java.util.*;
public class SelectionSort {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter size of array: ");
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.println("Enter elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        selectionSort(arr, n);
        System.out.println("Sorted array:");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
    }
    static void selectionSort(int[] arr, int n) {
        for (int i = 0; i < n - 1; i++) {
            int minIdx = i;
            // Find the index of the minimum element
            for (int j = i + 1; j < n; j++) {
                if (arr[j] < arr[minIdx]) {
                    minIdx = j;
                }
            }
            // Swap the found minimum element with the first element
            int temp = arr[i];
            arr[i] = arr[minIdx];
            arr[minIdx] = temp;
        }
    }
}
```

* **Code Logic Explanation:**

1. Start from the first element and assume it is the minimum.
2. Compare it with the remaining elements to find the actual minimum.
3. Swap the minimum element with the first unsorted element.
4. Move to the next position and repeat the same process.
5. Continue until the entire array is sorted.

* **Output:**

```
Enter size of array: 5
Enter elements:
 64   25   12   22   11
Sorted array:
 11   12   22   25   64
```

```
Passes (for above example):
Pass 1: 11  25  12  22  64
Pass 2: 11  12  25  22  64
Pass 3: 11  12  22  25  64
Pass 4: 12  22  22  25  64 (no swap)
        ⇒ Sorted
```

* **Time Complexity:** $O(n^2)$ | **Space Complexity:** $O(1)$

143. INSERTION SORT

* Question:

Write a program to sort an array using Insertion Sort algorithm in ascending order.

* Code (Java):

```
import java.util.*;
public class InsertionSort {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter size of array: ");
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.println("Enter elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        insertionSort(arr, n);
        System.out.println("Sorted array:");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
    }
    static void insertionSort(int[] arr, int n) {
        for (int i = 1; i < n; i++) {
            int key = arr[i];
            int j = i - 1;
            // Move elements of arr[0..i-1], that are greater
               than key, to one position ahead
            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                j = j - 1;
            }
            arr[j + 1] = key;  // Place key at correct position
        }
    }
}
```

* Code Logic Explanation:

1. Start from the second element (index 1) and take it as the key.
2. Compare the key with elements on its left.
3. Shift all greater elements one position to the right.
4. Place the key in its correct position.
5. Repeat for all elements until the array is sorted.

* Output:

```
Enter size of array: 5
Enter elements:
12  11  13  5  6
Sorted array:
5  6  11  12  13
```

Passes (for above example):

Pass 1:	11	12	13	5	6	
Pass 2:	11	12	13	5	6	
Pass 3:	5	11	12	13	6	
Pass 4:	5	6	11	12	13	⇒ Sorted

* Time Complexity: $O(n^2)$ | Space Complexity: $O(1)$

144. MERGE SORT

* **Question:**

Write a program to sort an array using Merge Sort algorithm in ascending order.

* **Code (Java):**

```java
import java.util.*;
public class MergeSort {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter size of array: ");
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.println("Enter elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        mergeSort(arr, 0, n - 1);
        System.out.println("Sorted array: ");
        for (int x : arr) System.out.print(x + " ");
    }
    static void mergeSort(int[] arr, int l, int r) {
        if (l < r) {
            int m = l + (r - l) / 2;      // find middle point
            mergeSort(arr, l, m);         // sort left half
            mergeSort(arr, m + 1, r);     // sort right half
            merge(arr, l, m, r);          // merge the sorted halves
        }
    }
    static void merge(int[] arr, int l, int m, int r) {
        int n1 = m - l + 1;
        int n2 = r - m;
        int[] L = new int[n1];
        int[] R = new int[n2];
        for (int i = 0; i < n1; i++) L[i] = arr[l + i];
        for (int j = 0, j < n2; j++) R[j] = arr[m + 1 + j];
        int i = 0, j = 0, k = l;
        while (i < n1 && j < n2) {
            if (L[i] <= R[j]) arr[k++] = L[i++];
            else arr[k++] = R[j++];
        }
        while (i < n1) arr[k++] = L[i++];  // copy remaining of L
        while (j < n2) arr[k++] = R[j++];  // copy remaining of R
    }
}
```

* **Code Logic Explanation:**

1. If the array has more than one element, find the middle point.
2. Recursively sort the left half and the right half.
3. Merge the two sorted halves into a single sorted array.
4. The merge process compares elements from both halves and places the smaller element into the original array.
5. Repeat until the entire array is sorted.

* **Output:**

```
Enter size of array: 6
Enter elements:
38 27 43 3 9 82
Sorted array:
3 9 27 38 43 82
```

Passes (for above example):

Initial: 38 27 43 3 9 82
After pass 1 (merge pairs): 27 38 3 43 9 82
After pass 2: 3 27 38 43 9 82
After pass 3 (merge all): 3 9 27 38 43 82
⇒ Sorted

* **Time Complexity:** $O(n \log n)$ | **Space Complexity:** $O(n)$

145. QUICK SORT

* **Question :**

Write a program to sort an array using Quick Sort algorithm in ascending order.

* **Code (Java) :**

```
import java.util.*;
public class QuickSort {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter size of array: ");
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.println("Enter elements:");
        for (int i = 0; i < n; i++)
            arr[i] = sc.nextInt();

        quickSort(arr, 0, n - 1);
        System.out.println("Sorted array:");
        for (int x : arr) System.out.print(x + " ");
    }
    static void quickSort(int[] arr, int low, int high) {
        if (low < high) {
            int pi = partition(arr, low, high);    // pivot index
            quickSort(arr, low, pi - 1);           // left part
            quickSort(arr, pi + 1, high);          // right part
        }
    }
    static int partition(int[] arr, int low, int high) {
        int pivot = arr[high];  // choose last element as pivot
        int i = low - 1;        // index of smaller element
        for (int j = low; j < high; j++) {
            if (arr[j] <= pivot) {
                i++;
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;    // swap
            }
        }
        int temp = arr[i + 1];
        arr[i + 1] = arr[high];   // place pivot
        arr[high] = temp;
        return i + 1;             // return pivot index
    }
}
```

* **Code Logic Explanation :**

1. Choose the last element as pivot.
2. Rearrange the array so that all elements smaller than or equal to pivot are on the left, and larger elements are on the right.
3. The pivot is now in its correct sorted position.
4. Recursively apply the same steps to the left and right sub-arrays.
5. Repeat until the entire array is sorted.

* **Output :**

```
Enter size of array: 6
Enter elements:
10  7  8  9  1  5
Sorted array:
1  5  7  8  9  10
```

```
Passes (for above example):
Pass 1: pivot=5 →   1  5  8  9  10  7   (pi=1)
Pass 2: left part [1] → already sorted
Pass 3: right part [8  9  10  7], pivot=7
                ⇒ 1  5  7  9  10  8   (pi=2)
Pass 4: sort [9  10  8], pivot=8
                → 1  5  7  8  10  9   (pi=3)
Pass 5: left [9], right [10] ⇒ sorted
Final Sorted:  1  5  7  8  9  10 ☑
```

* **Time Complexity :** $O(n \log n)$ (avg) | **Worst Case :** $O(n^2)$ | **Space Complexity :** $O(\log n)$

146. DFS TRAVERSAL

* **Question:**

Write a program to perform Depth First Search (DFS) traversal of a graph starting from a given vertex.

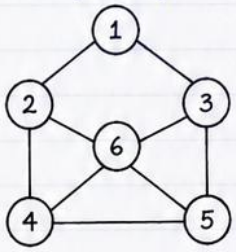
* **Code (Java):**

```java
import java.util.*;
public class DFSTraversal {
    static void dfs(int v, boolean[] visited, ArrayList<Integer>[] adj) {
        visited[v] = true;              // mark current node as visited
        System.out.print(v + " ");      // print the node
        for (int nbr : adj[v]) {
            if (!visited[nbr])
                dfs(nbr, visited, adj);   // visit unvisited neighbour
        }
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of vertices: ");
        int V = sc.nextInt();
        ArrayList<Integer>[] adj = new ArrayList[V];
        for (int i = 0; i < V; i++) adj[i] = new ArrayList<>();
        System.out.print("Enter number of edges: ");
        int E = sc.nextInt();
        System.out.println("Enter edges (u v): ");
        for (int i = 0; i < E; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();
            adj[u].add(v);
            adj[v].add(u);      // for undirected graph
        }
        boolean[] visited = new boolean[V];
        System.out.print("Enter starting vertex: ");
        int start = sc.nextInt();
        System.out.print("DFS Traversal: ");
        dfs(start, visited, adj);
    }
}
```

* **Code Logic Explanation:**

1. Use Adjacency List to represent the graph.
2. Create a visited[] array to track visited vertices.
3. Start DFS from the given starting vertex.
4. Mark the current vertex as visited and print it.
5. For each unvisited neighbour, recursively call DFS.
6. The process continues until all reachable vertices are visited.

Example Graph:



Adjacency List:
1: 2, 3
2: 1, 4, 6
3: 1, 5, 6
4: 2, 5, 6
5: 3, 4, 6
6: 2, 3, 4, 5

* **Output:**

Sample Input
```
Enter number of vertices: 6
Enter number of edges: 7
Enter edges (u v):
1 2
1 3
2 4
3 5
2 6
3 6
4 5
Enter starting vertex: 1
```

Sample Output
```
DFS Traversal: 1 2 4 5 3 6
```

Explanation of Output:

Starting from 1, we first go to 2. From 2, we go to 4, then 5, then backtrack and go to 3, and finally visit 6. (One possible DFS order.)

* **Time Complexity:** $O(V + E)$ | **Space Complexity:** $O(V)$

147. BFS TRAVERSAL

* **Question :**

Write a program to perform Breadth First Search (BFS) traversal of a graph starting from a given vertex.

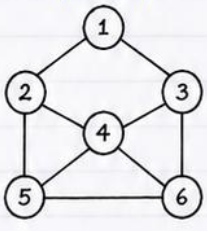
* **Code (Java) :**

```java
import java.util.*;
public class BFSTraversal {
    static void bfs(int start, boolean[] visited, ArrayList<Integer>[] adj) {
        Queue<Integer> q = new LinkedList<>();  // create queue
        visited[start] = true;                  // mark start as visited
        q.add(start);                           // add start to queue
        System.out.print("BFS Traversal: ");
        while (!q.isEmpty()) {
            int v = q.poll();                   // remove front of queue
            System.out.print(v + " ");          // print the vertex
            for (int nbr : adj[v]) {
                if (!visited[nbr]) {
                    visited[nbr] = true;        // mark neighbour visited
                    q.add(nbr);                 // enqueue neighbour
                }
            }
        }
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of vertices: ");
        int V = sc.nextInt();
        ArrayList<Integer>[] adj = new ArrayList[V];
        for (int i = 0; i < V; i++) adj[i] = new ArrayList<>();
        System.out.print("Enter number of edges: ");
        int E = sc.nextInt();
        System.out.println("Enter edges (u v): ");
        for (int i = 0; i < E; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();
            adj[u].add(v);
            adj[v].add(u);      // undirected graph
        }
        boolean[] visited = new boolean[V];
        System.out.print("Enter starting vertex: ");
        int start = sc.nextInt();
        bfs(start, visited, adj);
    }
}
```

* **Code Logic Explanation :**

1. Use a queue to explore vertices level by level.
2. Start from the given vertex, mark it visited and enqueue it.
3. Dequeue a vertex, print it, and enqueue all its unvisited neighbours.
4. Repeat until the queue is empty.
5. This gives the BFS traversal order.

Example Graph :



Adjacency List:
1: 2, 3
2: 1, 4, 5
3: 1, 4, 6
4: 2, 3, 5, 6
5: 2, 4
6: 3, 4

* **Output :**

Sample Input

```
Enter number of vertices: 6
Enter number of edges: 7
Enter edges (u v):
1 2
1 3
2 4
3 4
2 5
3 6
4 5
Enter starting vertex: 1
```

Sample Output

```
BFS Traversal: 1 2 3 4 5 6
```

Explanation of Output :

Start at 1.
Visit 1's neighbours → 2, 3.
From 2 add 4, 5 to queue; from 3 add 6.
Process 4 (no new nodes), then 5, then 6.
So BFS order is 1 2 3 4 5 6.

* **Time Complexity :** $O(V + E)$ | **Space Complexity :** $O(V)$

148. FIBONACCI USING DP

* **Question :**

Write a program to find the nth Fibonacci number using Dynamic Programming (Bottom-Up).

* **Code (Java) :**

```java
import java.util.*;
public class FibonacciDP {
    public static long fib(int n) {
        if (n <= 1) return n;          // Base cases
        long[] dp = new long[n + 1]; // dp[i] stores Fibonacci(i)
        dp[0] = 0;
        dp[1] = 1;
        for (int i = 2; i <= n; i++) { // Build table bottom-up
            dp[i] = dp[i-1] + dp[i-2];
        }
        return dp[n];                  // nth Fibonacci number
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter n: ");
        int n = sc.nextInt();
        System.out.println("Fibonacci number " + n + " is: " + fib(n));
    }
}
```

* **Code Logic Explanation :**

1. Fibonacci sequence: 0, 1, 1, 2, 3, 5, 8, ...
2. We use an array dp[] where dp[i] stores Fibonacci number at index i.
3. Base cases:
   - dp[0] = 0
   - dp[1] = 1
4. For i from 2 to n:
   dp[i] = dp[i-1] + dp[i-2]
5. The answer is stored in dp[n].
6. This is Bottom-Up DP (Tabulation).

* **DP Table Build (Example n = 7):**

i	0	1	2	3	4	5	6	7
dp[i]	0	1	1	2	3	5	8	13
Formula	–	–	0+1	1+1	1+2	2+3	3+5	5+8

* **Complexity :**

Time Complexity : O(n)
Space Complexity : O(n)

* **Sample Input / Output :**

Input	Output
Enter n: 7	Fibonacci number 7 is: 13

* **More Examples :**

n	0	1	2	3	4	5	6	7	8	10
Fibonacci(n)	0	1	1	2	5	8	13	21	34	55

* **Note:** DP avoids recomputation of the same subproblems.
Far more efficient than naive recursion (which is exponential).

149 0/1 Knapsack Problem

Q. Given weights and values of n items, put these items in a knapsack of capacity W to get the maximum total value.

Note: In 0/1 Knapsack, each item can be taken only once.

Example: Values : [60, 100, 120]
Weights : [10, 20, 30]
Capacity, W = 50
Maximum Value = 220

Code (Java):

```java
import java.util.*;
public class Knapsack01 {
    // Recursive function with memoization (Top-Down DP)
    static int knapSack(int W, int wt[], int val[], int n, int[][] dp) {
        if (n == 0 || W == 0)  return 0;            // Base case
        if (dp[n][W] != -1)  return dp[n][W];   // Already computed
        if (wt[n-1] > W)
            return dp[n][W] = knapSack(W, wt, val, n-1, dp);   // Don't take
        else {
            int take = val[n-1] + knapSack(W-wt[n-1], wt, val, n-1, dp);  // Take
            int notTake = knapSack(W, wt, val, n-1, dp);                  // Not take
            return dp[n][W] = Math.max(take, notTake);
        }
    }
    public static void main(String[] args) {
        int val[] = {60, 100, 120};
        int wt[] = {10, 20, 30};
        int  W = 50;
        int  n = val.length;
        int[][] dp = new int[n+1][W+1];
        for(int[] row : dp) Arrays.fill(row, -1);
        int maxValue = knapSack(W, wt, val, n, dp);
        System.out.println("Maximum Value = " + maxValue);
    }
}
```

Code Logic Explanation:

- This is a 0/1 Knapsack problem solved using Top-Down Dynamic Programming.
- We have a recursive function knapSack(W, wt, val, n, dp)
  where,
  - W → remaining capacity of knapsack
  - n → number of items considered
  - wt → array of weights
  - val → array of values
  - dp → memoization table of size (n+1) × (W+1)
- Base Case: If no items left (n == 0) or capacity is 0, return 0.
- If current item's weight is greater than W, we cannot take it.
- Otherwise, we have two choices:
  1. Take the item → value[n-1] + solve for remaining capacity (W - wt[n-1])
  2. Not take the item → solve for remaining items with same capacity (W)

  We return the maximum of both choices.
- dp[n][W] stores the result of subproblem to avoid recomputation.

Output:

```
Maximum Value = 220
```

Explanation of Output:

The items with weights 20 and 30 (values 100 and 120 respectively) give the maximum value 220 within capacity 50.

150 Sudoku Solver

Q. Solve the given 9x9 Sudoku puzzle.

Note: Empty cells are denoted by 0.

Example Input:

5	3	0	0	7	0	0	0	0
6	0	0	1	9	5	0	0	0
0	9	8	0	0	0	0	6	0
8	0	0	0	6	0	0	0	3
4	0	0	8	0	3	0	0	1
7	0	0	0	2	0	0	0	6
0	6	0	0	0	0	2	8	0
0	0	0	4	1	9	0	0	5
0	0	0	0	8	0	0	7	9

Goal:

Fill the empty cells (0) such that

- Each row contains 1-9
- Each column contains 1-9
- Each 3x3 subgrid contains 1-9

Code (Java):

```java
import java.util.*;
public class SudokuSolver {
    public static void main(String[] args) {
        int[][] board = {
            {5, 3, 0, 0, 7, 0, 0, 0, 0},
            {6, 0, 0, 1, 9, 5, 0, 0, 0},
            {0, 9, 8, 0, 0, 0, 0, 6, 0},
            {8, 0, 0, 0, 6, 0, 0, 0, 3},
            {4, 0, 0, 8, 0, 3, 0, 0, 1},
            {7, 0, 0, 0, 2, 0, 0, 0, 6},
            {0, 6, 0, 0, 0, 0, 2, 8, 0},
            {0, 0, 0, 4, 1, 9, 0, 0, 5},
            {0, 0, 0, 0, 8, 0, 0, 7, 9}
        };
        solveSudoku(board);
        printBoard(board);
    }
    static boolean solveSudoku(int[][] board) {
        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                if (board[row][col] == 0) {
                    for (int num = 1; num <= 9; num++) {
                        if (isSafe(board, row, col, num)) {
                            board[row][col] = num;
                            if (solveSudoku(board))
                                return true; // move forward
                            board[row][col] = 0; // backtrack
                        }
                    }
                    return false; // trigger backtrack
                }
            }
        }
        return true; // solved
    }

    static boolean isSafe(int[][] board, int row,
        int col, int num) {
        // check row and column
        for (int i = 0; i < 9; i++)
            if (board[row][i] == num || board[i][col] == num)
                return false;
        // check 3x3 subgrid
        int startRow = row - row % 3;
        int startCol = col - col % 3;
        for (int i = 0; i < 3; i++)
            for (int j = 0; j < 3; j++)
                if (board[startRow+i][startCol+j] == num)
                    return false;
        return true;
    }
    static void printBoard(int[][] board) {
        for (int i = 0; i < 9; i++) {
            for (int j = 0; j < 9; j++)
                System.out.print(board[i][j] + " ");
            System.out.println();
            if ((i+1) % 3 == 0 && i != 8) System.out.println();
        }
    }
}
```

Code Logic Explanation:

- We use Backtracking to solve the Sudoku.
- Find an empty cell (0).
- Try numbers from 1 to 9.
- If a number is safe (not present in the same row, column and 3x3 box), place it.
- Recur for the next empty cell.
- If no number fits, backtrack (reset to 0) and try the next possibility.
- If all cells are filled, return true.

isSafe() checks:

1. Row - number not in the row.
2. Column - number not in the column.
3. 3x3 Subgrid - number not in the box.

Output:

Solved Sudoku Puzzle:

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

Time Complexity:

- In worst case: $O(9^{(N)})$ where N = number of empty cells (backtracking).

Space Complexity:

- O(N) for recursion stack.

Backtracking is the key to solving Sudoku !